
Programming in C

Variables,
Operators,
Selection statements,
Repetition statements,
Functions,
Iteration vs. Recursion,
C preprocessor

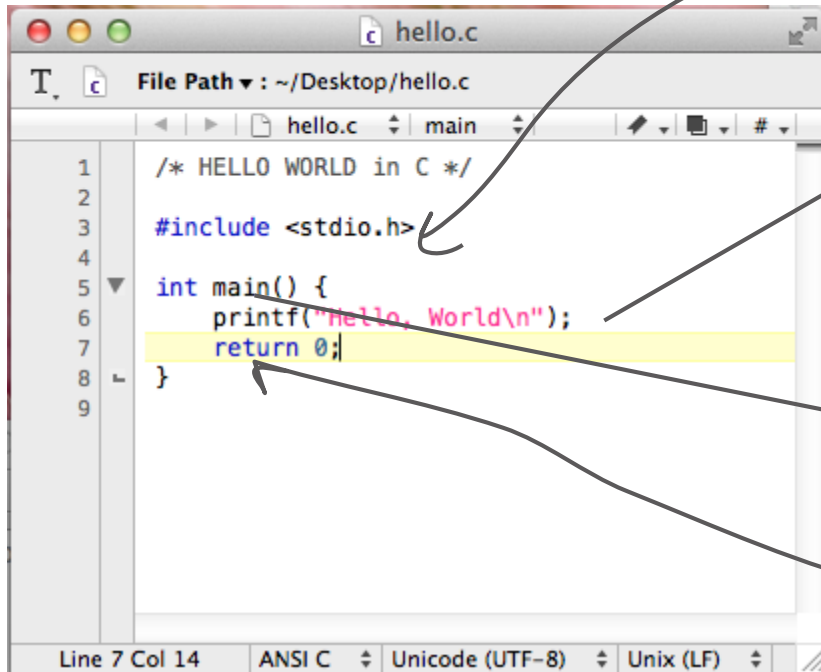
Previously on CE316

- “Hello, World” in C; did you try it at home?
 - Any questions before we go any further?
-

Hello, World

- Welcome back to programming in C!
 - Remember “Hello, World”
 - The goal (or purpose) of the program was to print “Hello, World” on the screen. To do so, we had to use the printf function, which lives in the stdio.h file.
 - Since the starting point of a program is main, the resulting program was as follows:
-

Hello, World



```
1  /* HELLO WORLD in C */
2
3
4  #include <stdio.h>
5
6  int main() {
7      printf("Hello, World\n");
8      return 0;
9  }
```

The screenshot shows a code editor window titled 'hello.c'. The file path is '~/Desktop/hello.c'. The code is a simple C program that prints 'Hello, World' to the screen. The code is as follows:

```
1  /* HELLO WORLD in C */
2
3
4  #include <stdio.h>
5
6  int main() {
7      printf("Hello, World\n");
8      return 0;
9  }
```

Arrows from the explanatory text point to the following elements in the code:

- An arrow points from the text 'The goal of program is to print "Hello, World on screen, so we have to use printf which lives in stdio.h' to the `#include <stdio.h>` line.
- An arrow points from the text 'main is the entry point for all C programs. It returns an integer (int).' to the `int main() {` line.
- An arrow points from the same text to the `return 0;` line.

The goal of program is to print "Hello, World on screen, so we have to use printf which lives in stdio.h

main is the entry point for all C programs. It returns an integer (int).

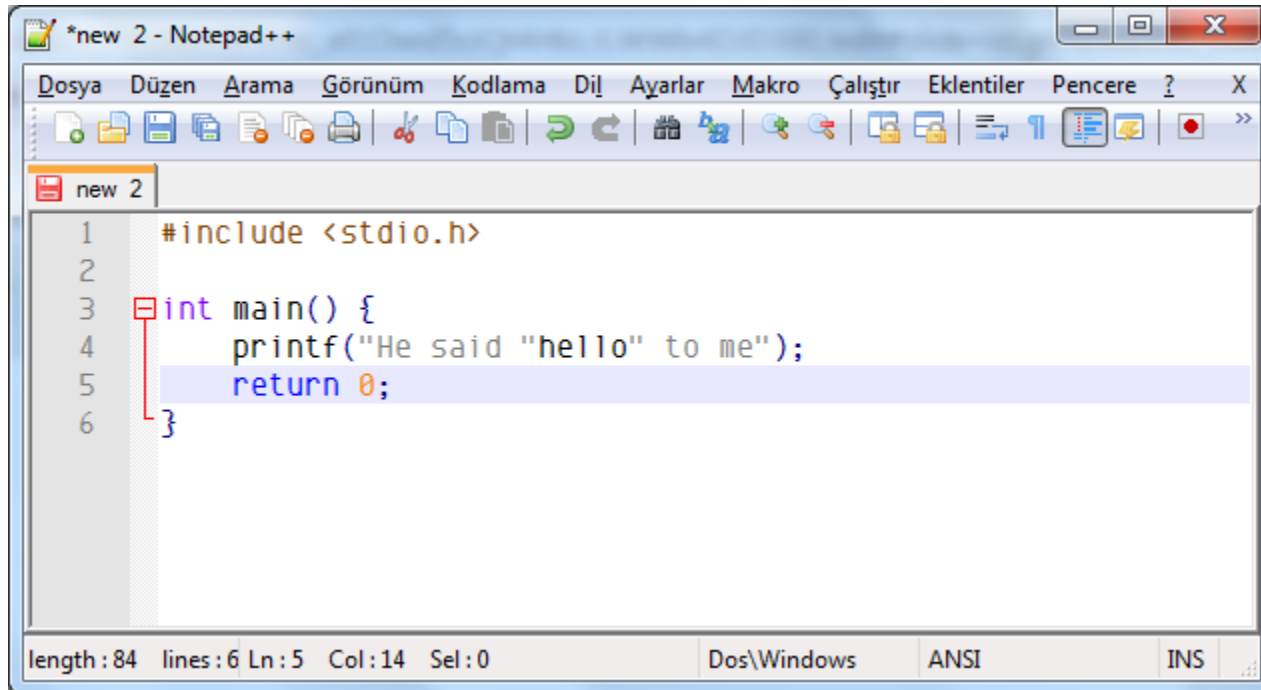
printf

- Remember that I have told you “printf” does not care what you write between double quotes.
 - Actually, it does. But cares only for some elements that need formatting.
 - Also, remember the new line character: `\n`
 - The backslash preceding the character `n` is called the “escaping character”
 - What is escaping?
-

printf

- Suppose, you want to print the following sentence on screen:
He said “Hello” to me.
 - If you use double quotes around Hello, then how can printf know that the parameter is complete.
 - Look at the syntax highlighting in the text editor:
-

Escaping



The screenshot shows a Notepad++ window titled '*new 2 - Notepad++'. The menu bar includes 'Dosya', 'Düzen', 'Arama', 'Görünüm', 'Kodlama', 'Dil', 'Ayarlar', 'Makro', 'Çalıştır', 'Eklentiler', 'Pencere', '?', and 'X'. The toolbar contains various icons for file operations and editing. The editor shows a C program with the following code:

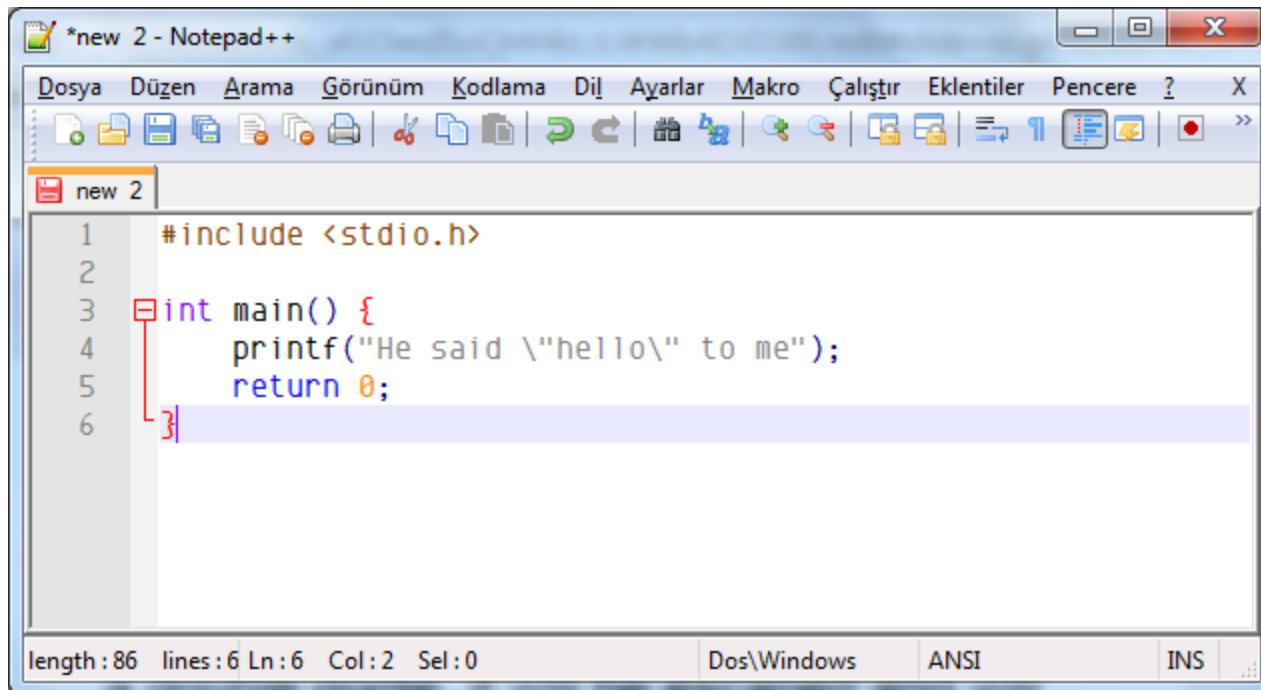
```
1 #include <stdio.h>
2
3 int main() {
4     printf("He said "hello" to me");
5     return 0;
6 }
```

The code is highlighted in blue. The status bar at the bottom shows 'length: 84 lines: 6 Ln: 5 Col: 14 Sel: 0' and the file path 'Dos\Windows' with encoding 'ANSI' and cursor position 'INS'.

Clearly, there is a problem when you want to use double quotes within printf.

Escaping

- As it is with `\n`, you can use backslash before a double quote, it will be escaped and will not be considered as the end of parameter.
 - What will the program output?
 - Try it! Let's call it your first homework for this week.
-



```
*new 2 - Notepad++
Dosya Düzen Arama Görünüm Kodlama Dil Ayarlar Makro Çalıştır Eklentiler Pencere ? X

new 2
1 #include <stdio.h>
2
3 int main() {
4     printf("He said \"hello\" to me");
5     return 0;
6 }
```

length: 86 lines: 6 Ln: 6 Col: 2 Sel: 0 Dos\\Windows ANSI INS

Escaping

- What if you want to use backslash?
 - Then, you escape backslash with backslash: `\\`
 - We will mostly use escaping for new lines (`\n`) and using double quotes within `printf`.
 - Here is a list of other special characters:
 - `\t` → tab (to align your output)
 - `\a` → makes the computer go BEEP! (alert)
 - They look like two characters but they represent only one.
-

Variables

- Variables and constants are the basic data manipulated in a program.
 - Variables are containers for your data.
 - However, C needs to know what kind of data you want to store. So, it has a list of types.
 - You already have seen **int** variable type from the main function. Let's take a closer look at them.
-

Variables

int	Integers
char	Characters
float	Floating point (single precision)
double	Floating point (double precision)

Variables

- Besides its type, you also should give your variable a name (called identifier).
 - Naming a variable is easy:
 - The name must start with a character or `_`
 - The following can be characters, numbers or `_`
 - The following are valid variable names: `num1`, `grade`
 - Variable names are case sensitive; that means **grade** and **GRADE** are different variables.
 - Naming your variables: try to choose meaningful, concise variable names.
-

Variables

- What can we do with variables?
 - Why do we need them?
 - You can use them to store data.
 - Otherwise, you have to use constant values all the time.
 - What is the meaning of programming if we can not program for different values?
-

Variables

- As you may have noticed, three out of four variable types are related to numbers.
 - Only **char** type is related to characters. Remember, though, characters were also represented by numbers.
 - Before we use C like a simple calculator, let's learn more about variables.
-

Variables

- All variables must be **declared** before use.
 - All variables are **defined** in the memory depending on their scope in the program.
 - A **declaration** specifies a type, and contains a list of one or more variables of that type.
 - A variable may also be initialized in its declaration.
 - Actually you should always try to initialize your variables.
-

var01.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Management

Projects Symbols

Workspace

Start here x var01.c x

```
1  /* This file adds two numbers */
2  /* Version 0.1 */
3
4  #include <stdio.h>
5
6  int main() {
7      int num1 = 12;
8      int num2 = 23;
9      int num3 = num1 + num2;
10
11     printf("How can I print numbers?");
12     return 0;
13 }
14
15
```

Windows (CR+LF) UTF-8 Line 12, Column 14 Insert Read/Write default

printf: revisited

- We want to be able to display the variable value.
- To do so, we use placeholders as “format specifiers” that start with %
- For integers (int) we use **d** (for **d**ecimal).
- So far, printf had only one argument (the double quoted string).
- If we use %d inside this first argument, it means that when output on screen it will be replaced by the second parameter, which is likely an integer.
- The parameters are separated by a comma (,).

var01.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Management

Projects Symbols

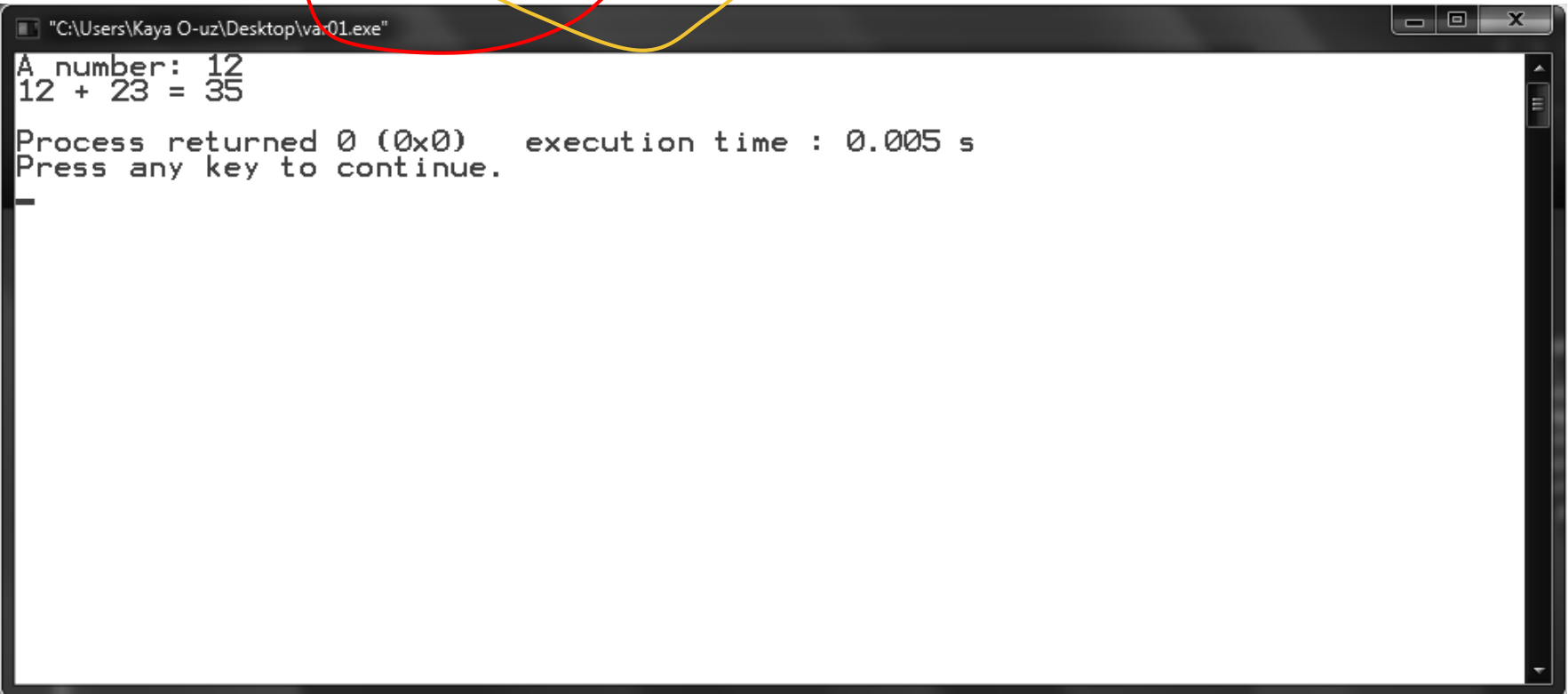
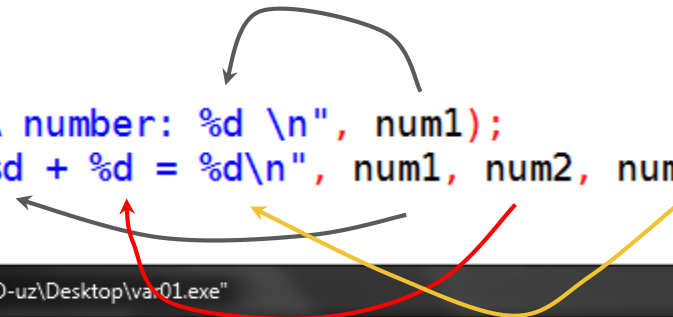
Workspace

Start here x var01.c x

```
1  /* This file adds two numbers */
2  /* Version 0.2 */
3
4  #include <stdio.h>
5
6  int main() {
7      int num1 = 12;
8      int num2 = 23;
9      int num3 = num1 + num2;
10
11     printf("A number: %d \n", num1);
12     printf("%d + %d = %d\n", num1, num2, num3);
13     return 0;
14 }
15
16
```

C:\Users\ Windows (CR+LF) UTF-8 Line 16, Column 1 Insert Read/Write default

```
printf("A number: %d \n", num1);  
printf("%d + %d = %d\n", num1, num2, num3);
```



```
"C:\Users\Kaya O-uz\Desktop\va01.exe"  
A number: 12  
12 + 23 = 35  
  
Process returned 0 (0x0)   execution time : 0.005 s  
Press any key to continue.  
_
```

Notice that the specifiers with %d are replaced with the value of corresponding variables

Before...

- ...we move on to variable types in detail, let's take a closer look to the previous source code.
 - The reason is, we have used many *operators* in there. Since they look familiar, you may have not noticed them. But we have assignment (=) and an operation (+).
-

Tokens in C

- There are 6 kinds of tokens (symbols) in C
 - keywords
 - identifiers
 - constants
 - string constants
 - punctuators
 - operators
 - Let's take a look at each of them.
-

Keywords

- Keywords are reserved for C: you can not use them as identifiers (such as variable names.)
- There are 32 of them. See below.
- Did you recognize some of them?

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

Identifiers

- Used to name variables and functions
- Case sensitive!
 - all different identifiers:
ilker
ILKER
IlKeR

Constants

- There are a few types of constants:
 - integer constants: e.g. 19 and 200
 - floating constants: e.g. 1.9 and 3.14
 - long constants: e.g. 123456789L
 - character constants: 'i', 'l', '+'
notice the **single quotes** around the character.
 - string constants: "ilker", "kaya", "damla"
A sequence of characters enclosed in a pair of **double quotes**.
"" is a null string
'a' is not the same as "a"
-

Punctuators

- We have used all of these:
 - Curly braces {}
 - Comma ,
 - Semicolon ;
 - Parentheses ()
-

Operators

- There are many operators in C. For now, the following is a small list of them:
 - `()` are used immediately after functions
 - `+`, `-`, `*`, `/` the mathematical operators for addition, subtraction, multiplication and division.
 - `%` integer modulo division (remainder)
 - `++`, `--` increment and decrement operator
 - `=`, `+=`, `-=`, `*=`, `/=`, `%=` assignment operators.
-

Operators

We'll talk more about precedence next week.

- Operators have precedence among themselves. For example, for a statement such as follows
`int a = 3 + 5 * 2;`
the multiplication has precedence over addition. So, first $5*2$ is evaluated.
 - Similar to mathematics, parentheses have higher precedence:
`int a = (3 + 5)*2`
In this case the addition inside parentheses is evaluated first.
-

Assignment

- Assignment is very important in C programming.
 - The value on the right hand side of the statement is assigned to the variable on the left:
`int a = 3;`
`int b = a + 4;`
 - In the first statement, the value 3 (integer constant) is assigned to variable a.
 - In the second statement, the expression `a+4` is evaluated first and the result is assigned to b.
-

Other Assignment Operators

- The equal sign (=) is the basic assignment operator. For example,
 `int x = 10;`
 `x = x + 3;`
 The final value of x is 13.
 - In mathematics an equation such as $x = x + 3$ is confusing. The x's on the both sides of the equation cancel each other.
 - **However, this is not an equation.** It is an assignment.
 - It means, assign the value of right hand side to the variable on the left hand side.
-

Other Assignment Operators

- If you use assignments, consider the following short hand syntax:
 - `x += 3; // means  $x = x + 3$ ;`
 - `x -= 3; // means  $x = x - 3$ ;`
 - `x *= 3; // means  $x = x * 3$ ;`
 - `x /= 3; // means  $x = x / 3$ ;`
- There is another handy operator, called increment/decrement operator:
 - `x++; // means  $x = x + 1$ ;`
 - `x--; // means  $x = x - 1$ ;`
 - So where does the name C++ come from? 😊

More operators

- Modulo operator gives you the remainder from the division. For example
`int x = 12;`
`int y = x % 5; // assigns 2 to y.`
`int z = x % 4; // assigns 0 to z.`
-

char

- Now we can go back to the variables.
 - The char variable has the smallest range. It only holds integer values between 0 to 255 (2^8) that correspond to a character.
 - For example, integer 65 is A and integer 97 is a.
 - Since it is only 8 bits long, it is only one byte.
 - So if you have a text file that is 900 bytes, it means there are 900 characters in it.
 - You can use “%c” in printf for characters.
-

char

- It is very important for you to notice that we are talking about single characters, not a list of characters or words.
 - A list of characters in C is called a string, which is an array of characters containing a special termination character '\0' at the end.
-

int

- The int type represents integers.
- There are many integer types.
- The integer type int supports both signed and unsigned integers.
- Unsigned integers are always positive.
- Signed integers (default int) can have negative values.
- You can try **sizeof(type)** to see the required memory area for a particular type in C.
- Most likely it uses 4 bytes, which means 2^{32} different values.
- You can use %d in printf

int

- In addition, there are a number of *qualifiers* that can be applied to these basic types.
- `short` and `long` apply to integers:

```
short int sh;  
long int counter;
```

- The word `int` can be omitted in such declarations, and typically is.
-

float and double

- Floating point variables are represented by float and double types.
 - The float type uses 4 bytes (mostly), and you can use the f suffix:
`float x = 1.4f;`
 - The double type uses 8 bytes and you don't require any suffix.
 - The double type is more precise.
 - You can use %f for both of them in printf.
 - The type long double specifies extended-precision floating point.
-

C and types

- C is very strict about types.
 - It is even strict with constant types.
 - For example:
 `printf("%d", 3 / 5);`
 prints 0 on screen, why?
 - How about this?
 `printf("%f", 3 / 5);`
 - You need floating point values to get floating values.
 `printf("%f", 3.0 / 5.0);`
-

C and types

- So, what to do if you have an integer value as the sum of all grades and another integer value for the number of students in a class. How can you calculate the average grade?
 - You can use float values for the sum and number of student values, but it is not what we always want.
 - You can force the compiler to change the type.
-

Casting

- This type changing operation is **casting**. It is a little early to talk about it, but a simple demonstration would not hurt.
- So, if you have two integer values (either constants or variables) you can cast them (convert them) to another type as follows:

```
printf("%f", (float)3 / (float)5);
```

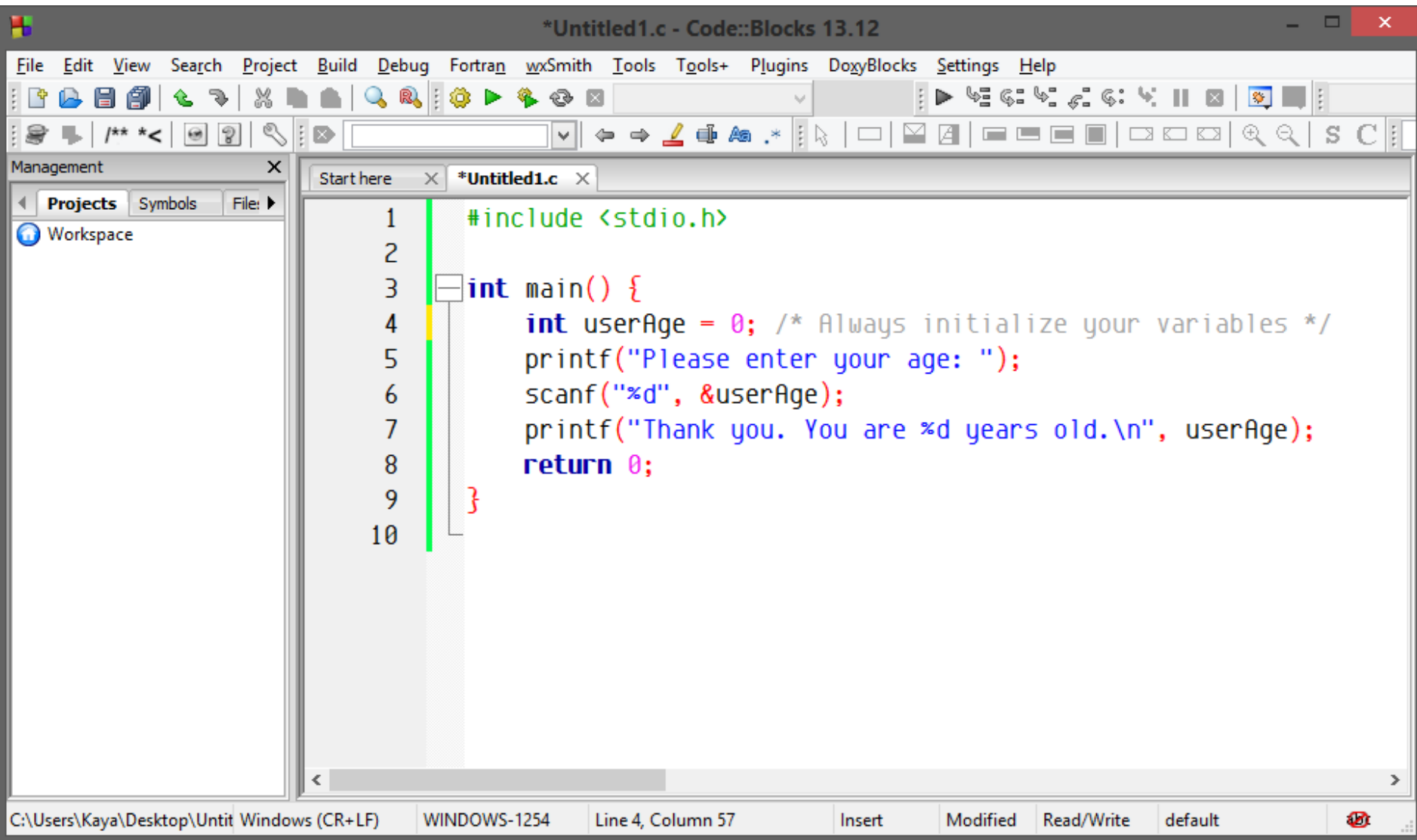
- This means, convert this constant value (or variable) to the type in the parentheses.
-

Casting

- Of course, some of these conversions are harmless.
 - But, what if you want to cast 3.2 to integer?
 - Then you lose precision, it becomes 3.
 - So, if you get weird 0 values, be sure to check your variable types.
-

Time for some input

- The printf function has a very close relative.
 - It is called **scanf**.
 - It uses similar format specifiers.
 - However, instead of outputting the value, it saves the data to the variable from an input source (your keyboard in this case).
 - So now, you can get some input from the user. Ask how old they are:
-



C:\Users\Kaya\Desktop\Untitled1.exe

Please enter your age: 34

Thank you. You are 34 years old.

Process returned 0 (0x0) execution time : 2.019 s

Press any key to continue.



- Please notice the ampersand (&) sign just before the variable name.
- When using **scanf** to save some data into a variable, you need this sign before the variable name (no-space between them).
- This means, “save the data referenced by the format specifier (%d) into this variable’s memory address”
- It is also your first glimpse to “pointers”

Variables and memory

- Everytime you declare an integer, C handles the definition of the data in memory for you.
For example:
`int x = 5;`
- A memory cell is created for variable x and the integer value 5 is saved there. The memory location depends on your operating system.
- You can think of memory as a very large file and you have been given an address of an integer size to hold your integer value.

Input

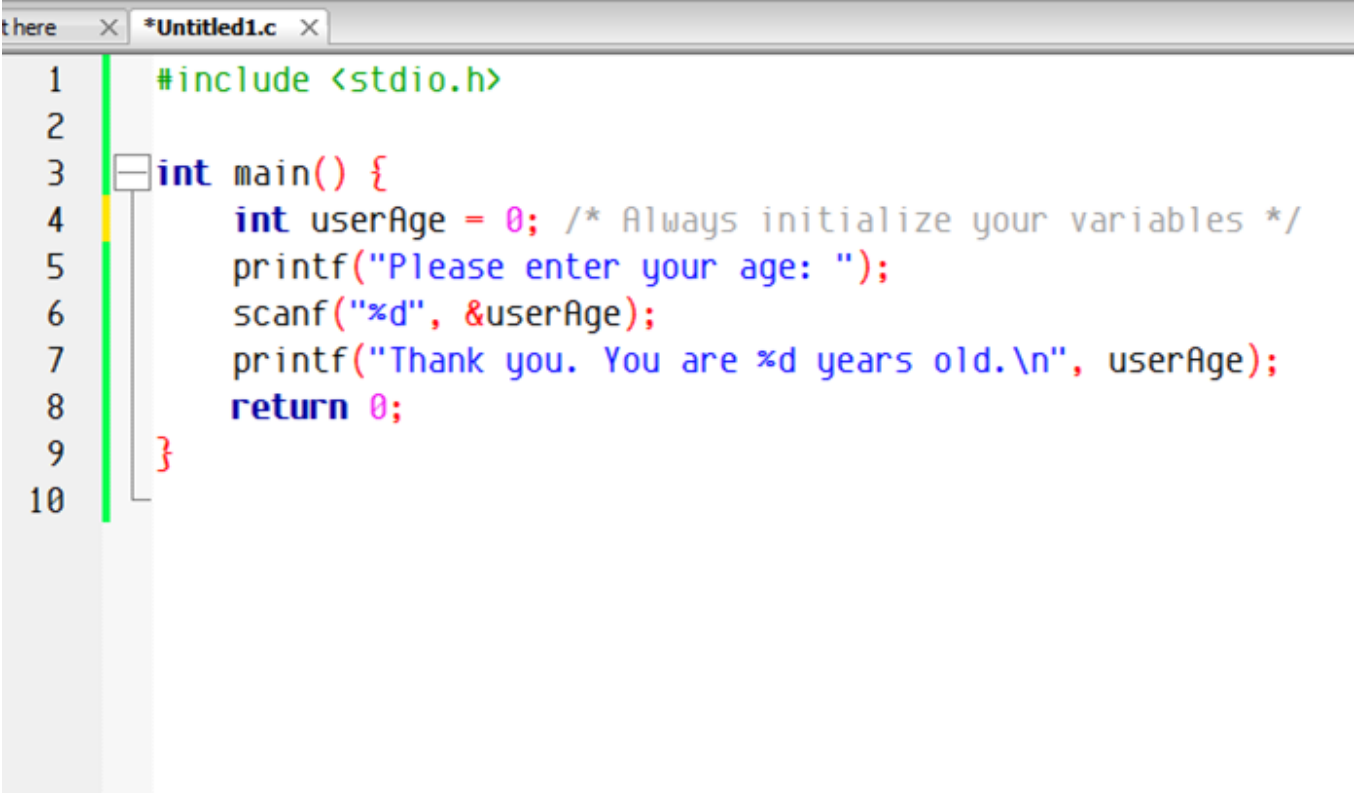
- The format specifier says that you are expecting an integer.
 - Here is a software engineering tip: Never trust the data that comes from the user.
 - People type all kinds of things.
-

scanf with floating point numbers

- You can use %f with scanf for float type.
 - With type double, you have to use %lf
 - At home, try to read floating point values and print them on screen.
-

The Flow

Top down
↓
flow



```
1  #include <stdio.h>
2
3  int main() {
4      int userAge = 0; /* Always initialize your variables */
5      printf("Please enter your age: ");
6      scanf("%d", &userAge);
7      printf("Thank you. You are %d years old.\n", userAge);
8      return 0;
9  }
10
```

The Flow

- A C program starts running from the _____ function.
 - It executes each expression step by step.
 - However, we can change the flow as required.
 - So far, each line executed one at a time.
 - How would a programmer know when to change the flow?
 - **If**, for example, the user has entered an invalid data.
-

Controlling User Input

- We use the **scanf** function to get a data from the user.
 - **If** we want to check this data, we can use operators, that look very much like mathematical operators.
 - Beware though, they only look like them, and in most cases, used like them.
-

Zero / Non-zero

- If you want to control a variable, say, if variable x equals 5;
 - it is either equal to 5, then the statement is TRUE
 - or it is not equal to 5, then the statement is FALSE
 - If the statement is TRUE, it corresponds to a non-zero value (mostly 1).
 - If the statement is FALSE, it corresponds to zero (0).
 - These values are called **boolean** values but they don't have a type in C. We use integers instead.
-

Be Logical

*Logic is the beginning of wisdom, not the end.
Mr. Spock.*

- Logical AND

- $T \text{ AND } T = T$
- $T \text{ AND } F = F$
- $F \text{ AND } T = F$
- $F \text{ AND } F = F$

- Logical OR

- $T \text{ OR } T = T$
- $T \text{ OR } F = T$
- $F \text{ OR } T = T$
- $F \text{ OR } F = F$



The Operators

Relational Operators		
Less than	<	x < 3;
Greater than	>	x > 4;
Less than or equal	<=	x <= 10;
Greater than or equal	>=	x >= 0;
Equality		
Equal	==	x == 2;
Not Equal	!=	x != 3;
Logical		
Negation	!	!(x == 2);
Logical AND	&&	(x != 2) && (x != 3);
Logical OR		(x == 1) (x == 4);

What did all that mean?

```
T. File Path ▾ : ~/true.c
true.c (no symbol selected)
1  #include <stdio.h>
2
3  int main() {
4      int x = 4;
5      printf("3 == 3 is %d\n", 3 == 3);
6      printf("x == 3 is %d\n", x == 3);
7      printf("x != 3 is %d\n", x != 3);
8      printf("x < 5 is %d\n", x < 5);
9      printf("x <= 9 is %d\n", x <= 9);
10     printf("Let's try logical operators\n");
11     printf("(x != 3) && (x != 1) is %d\n", (x!=3) && (x!=1));
12     /* can we store them in a variable?? */
13     int v = (x < 3) || (x >= 4);
14     printf("(x < 3) || (x >= 4) is %d\n", v);
15     return 0;
16 }
17
```

Check your answers

```
Kaya-iMac:~ koguz$ gcc true.c -o true
Kaya-iMac:~ koguz$ ./true
3 == 3 is 1
x == 3 is 0
x != 3 is 1
x < 5 is 1
x <= 9 is 1
Let's try logical operators
(x != 3) && (x != 1) is 1
(x < 3) || (x >= 4) is 1
Kaya-iMac:~ koguz$
```

A very common mistake is confusing the assignment operator = with the equal operator ==

The = operator assigns the value on the right to the variable on the left.

The == operator is used to compare its left and right operands.

- The ! negates the value; turns true into false and false into true.
- These are simple expressions.
- In complicated expressions, be careful with operator precedence.

The Precedence

- Now that we know a lot about operators, let's talk about precedence.
 - Similar to mathematics, C operates on expressions of varying complexity using a set of rules.
 - These rules define which operator comes first.
 - In mathematics, in an operation such as $3 + 4 \times 5$ the multiplication is done before the addition.
 - There is such a table in C, as well.
-

Precedence	Operator	Description	Associativity
1	++ -- () [] . -> (type){ list }	Suffix/postfix increment and decrement Function call Array subscripting Structure and union member access Structure and union member access through pointer Compound literal(C99)	Left-to-right
	++ -- + - ! ~ (type) * & sizeof _Alignof	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT Type cast Indirection (dereference) Address-of Size-of Alignment requirement(C11)	Right-to-left
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10		Bitwise OR (inclusive or)	
11	&&	Logical AND	
12		Logical OR	
13	? :	Ternary conditional	Right-to-Left
14	=	Simple assignment	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	&= ^= =	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

The Precedence

- The table on the previous slide is available from
http://en.cppreference.com/w/c/language/operator_precedence

Associativity

- Thank you for noticing associativity on the table.
 - It shows the order of evaluation with the operator.
 - In right to left associativity (like assignment operator), right is evaluated first.
 - So, you will never be confused again.
 - Let's test it.
-

Precedence and Associativity

- `int z = x > y;`
 - Easy. The relational operators are higher on the list, so `x > y` is evaluated first. The variable `z` gets true or false (1 or 0) value from the evaluation of `x > y`.
 - `z = x - y < 5;`
 - The minus operator ranks higher on the list. So, `x - y` is evaluated first. Then the `<` operator. Finally, the assignment.
 - `z = 3 < x < 5;`
 - Associativity! Operator `<` works left to right. So `3 < x` first, then the result (0 or 1) with `< 5`;
-

On Clarity and Readability

- The compiler does not make mistakes.
 - You are all familiar with the following term:
HUMAN ERROR
 - So, be clear when you are writing your code.
 - A statement such as
 $z = 3 < x < 5;$
does not confuse the compiler, but it might confuse a human (who is possibly reading and trying to find out the answer.)
 - So, use parentheses; readability makes you a better programmer. Remember that your code will be reviewed by your fellow engineers/programmers.
-

Let's test your logic

- `int x = 1;`
`int z = !x;`
`int t = !!x;`
 - What if x was 5 (x = 5) above?
 - This is a mathematical expression;
 $3 < x < 10$
which means x is between 3 and 10.
Here it is in C:
`(3 < x) && (x < 10);`
Why not use?
`3 < x < 10;`
-

C is clever: Short circuit

- The logical OR returns **true** as long as one of the operands is **true**.
 - We also know that in C, the OR operator `||` is evaluated left to right.
 - If the left operand returns **true**, do we need to check the one on the right?
 - No.
 - Neither does C.
 - Similarly, if the first operand of an AND (`&&`) is **false**, the second one is not checked (or evaluated).
-

Selection Statements

- So far, we have talked about operators, true/false values.
 - Let's talk about the flow using
 - if
 - switch
-

if statements

- If you want to change the flow of your program, you can use **if** statements.
- You need a true/false statement to check and accordingly you can run different codes.
- It is still written top-down :)
- Here is a simple case:

```
int x = 0; // always init. a variable.
scanf("%d", &x);
if(x == 13) {
    printf("is it your lucky number?");
}
```

if statements

- Consider the fragment below.

```
if (x == 13) {  
    printf("is it your lucky number?");  
}
```

- You know that the expression `x == 13` will return true(1) or false(0). If it returns true, then the `printf` expression will be evaluated. Otherwise, it will not be evaluated.
 - According to the user input, you have two different programs.
 - Notice the curly braces and the indentation.
-

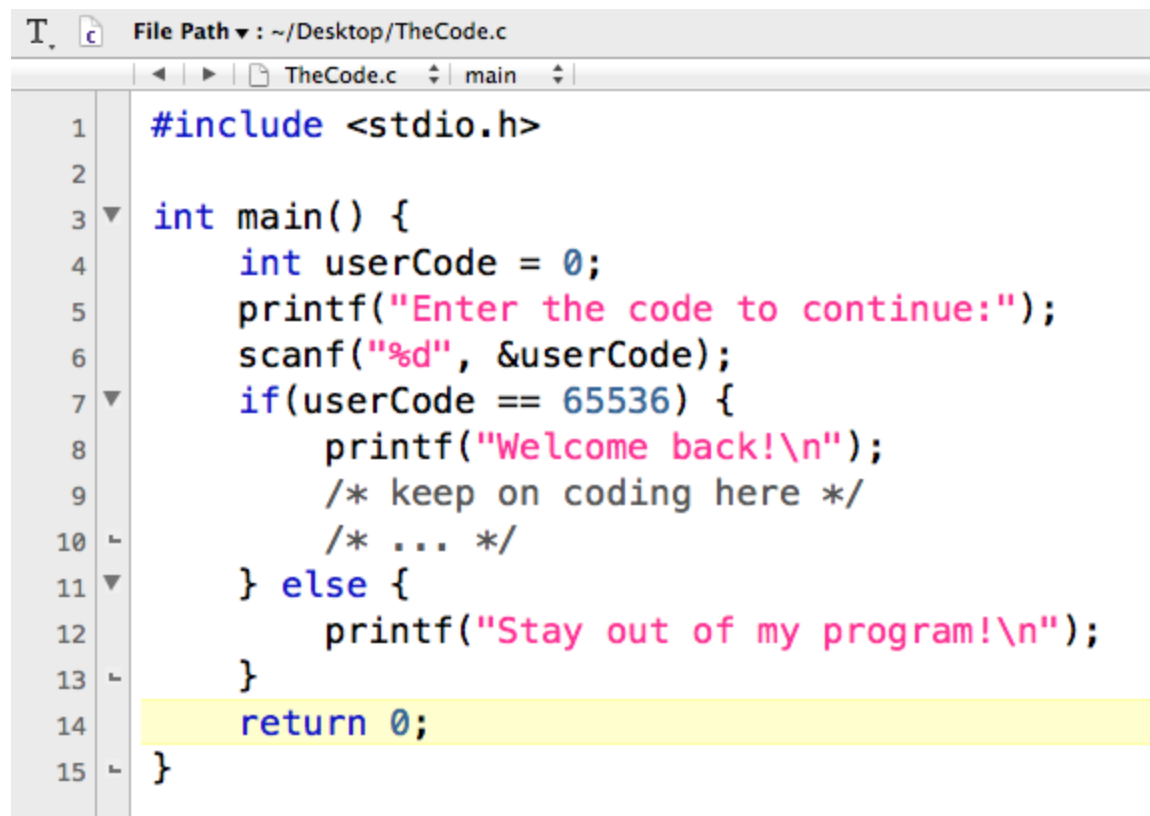
if statements

- Consider the fragment below

```
if (x == 13) {  
    printf("is it your lucky number?");  
}
```

- This piece of code is useful if you want to do something in only one case (in this case, if the user enters 13).
 - Let's try a secret code screen.
-

TheCode.c

A screenshot of a code editor window. The title bar shows 'T. c' and 'File Path: ~/Desktop/TheCode.c'. The editor displays a C program with line numbers 1 through 15 on the left. The code includes a header, a main function, variable declarations, printf and scanf calls, an if-else conditional, and a return statement. The line 'return 0;' is highlighted in yellow.

```
1  #include <stdio.h>
2
3  int main() {
4      int userCode = 0;
5      printf("Enter the code to continue:");
6      scanf("%d", &userCode);
7      if(userCode == 65536) {
8          printf("Welcome back!\n");
9          /* keep on coding here */
10         /* ... */
11     } else {
12         printf("Stay out of my program!\n");
13     }
14     return 0;
15 }
```

Stop... or else

- The user either enters the code correctly (true) or not (false).
 - For such cases you can use **else**.
 - Notice that the curly braces immediately follow the **else** keyword.
 - ```
if (...) {
 // run these statements if it is true
} else {
 // run these statements if it is false
}
```
-

# Here is a sample output

---

```
Kaya-iMac:Desktop koguz$ gcc TheCode.c -o TheCode
Kaya-iMac:Desktop koguz$./TheCode
Enter the code to continue:1231
Stay out of my program!
Kaya-iMac:Desktop koguz$./TheCode
Enter the code to continue:65536
Welcome back!
Kaya-iMac:Desktop koguz$ █
```

# Which one to use?

---

- It depends on your programming task.
  - Actually, there is one more type of these **if** statements where you can make it choose for more than two choices.
  - For such cases, we use **else if**. Typically, for creating menus on our command-line programs.
-



```
1 #include <stdio.h>
2
3 ▼ int main() {
4 int userCode = 0;
5 printf("Which one is your favorite color?\n");
6 printf("1. Red\n2. Blue\n3. Black\n4. White\n");
7 scanf("%d", &userCode);
8 ▼ if(userCode == 1) {
9 └─ printf("Red!\n");
10 ▼ } else if(userCode == 2) {
11 └─ printf("Blue is my favorite as well!\n");
12 ▼ } else if(userCode == 3) {
13 └─ printf("Black!\n");
14 ▼ } else if(userCode == 4) {
15 └─ printf("White\n");
16 ▼ } else {
17 printf("You should ask my programmer to add ");
18 printf("that color to the menu as well.\n");
19 └─ }
20 return 0;
21 └─ }
```

# else if

---

- It is really not that complicated:

```
if (...) {
 // run if this expression is true
} else if (...) {
 // run if this second expression is true
} else {
 // run if none of them are true
}
```

- You can skip the last **else**. In that case, if none of the statements return true, all of them are skipped.
  - You can have as much **else if** as you require.
-

# if inside an if? if it is possible, then...

---

- You can have another **if** statement in another **if** statement. We call them *nested* if statements.

```
if (expression1) {
 if (expression2) {
 statements
 }
}
```

- Of course, expression1 AND expression 2 must both be true for statements to run. Wait. AND?
-

```
#include <stdio.h>
```

```
int main() {
```

```
 int a;
```

```
 printf("Please enter a number: ");
```

```
 scanf("%d", &a);
```

```
 if (a%2 == 0) { // if the remainder is zero
```

```
 if(a%5 == 0) { // if the remainder is zero
```

```
 printf("%d is a multiple of 2 and 5.\n", a);
```

```
 }
```

```
 } else {
```

```
 /* notice that the indentation tells me
```

```
 this is the else for the first if */
```

```
 printf("%d is not a multiple of 2 and 5.\n", a);
```

```
 }
```

```
 if (a%2 == 0 && a%5 == 0) {
```

```
 printf("%d is a multiple of both 2 and 5.\n", a);
```

```
 } else {
```

```
 printf("%d is not a multiple of both 2 and 5.\n", a);
```

```
 }
```

```
 return 0;
```

```
}
```

NESTED

# What if?

---

- What if you enter 4 as input?
  - Homework: Write this program at home and explain why it gives different outputs.
-

# Conditional Operator

---

- You will be using **if** and **else** a lot. In some cases, it will only be as follows:

```
if (expression1) { expression2; }
else { expression3; }
```

- Well, there is a conditional operator which keeps it shorter:

```
expression1 ? expression2 : expression3;
```

---

# An example

---

```
if (x < 5) {
 y = 5;
} else {
 y = 10;
}
```

// this becomes:

```
y = (x < 5) ? 5 : 10;
```

// more fun:

```
printf("%d\n", (x < 5) ? 5 : 10);
```

---

# The switch statement

---

- There is a multiway conditional statement that generalizes the **if-else if-else** statements.
  - **switch** (expression) {  
    **case** expr1: /\* statement(s) \*/  
    **case** expr2: /\* statement(s) \*/  
    /\* as much case as you need \*/  
    **default**: /\* if none of them match, switch  
uses this instead \*/  
}
  - There is a catch with the **switch**.
-



# Moving along cases:

---

- ```
int x = 0;
scanf("%d", &x);
switch(x) {
    case 1:
        printf("entered 1");
    case 2:
        printf("entered 2");
    default:
        printf("entered something else");
}
```
 - The switch evaluates `x` and if it is 1, it ***starts*** evaluating from case 1. That means, it runs all the remaining (inc. default) as well.
 - If it is 2, it skips case 1 and starts from case 2.
-

switch is not the same as if

- So, switch is not the same as **if**.
 - However, we can make it run like **if**, if we want to.
 - We can use the keyword **break** to stop the cases when we are done.
 - The **break** keyword is also used in loops.
 - It causes an exit from the innermost (in this case it is nested) **switch**.
 - Here is the favourite colour program using **switch** (with **breaks** of course).
-

```
1  #include <stdio.h>
2
3  ▾ int main() {
4      int userCode = 0;
5      printf("Which one is your favorite color?\n");
6      printf("1. Red\n2. Blue\n3. Black\n4. White\n");
7      scanf("%d", &userCode);
8  ▾  switch(userCode) {
9      case 1:
10         printf("Red!\n");
11         break;
12     case 2:
13         printf("Blue!\n");
14         break;
15     case 3:
16         printf("Black!\n");
17         break;
18     case 4:
19         printf("White!\n");
20         break;
21     default:
22         printf("You should ask my programmer to add ");
23         printf("that color to the menu as well.\n");
24     }
25     return 0;
26 }
```

switch

- So, why need switch?
 - It has its uses. Consider a problem where you need the program to jump to a line according to the input.
 - See the code on the next slide.
-

switch put to good use

```
1  #include <stdio.h>
2
3  int main() {
4      int day = 0;
5      printf("What day is it (0-6): ");
6      scanf("%d", &day);
7      switch(day) {
8          case 0:
9          case 1:
10         case 2:
11         case 3:
12         case 4:
13             printf("It is a weekday\n");
14             break;
15         case 5:
16         case 6:
17             printf("Weekend!");
18             break;
19     }
20     return 0;
21 }
22
```

Using if or switch?

- You can use either **if** or a **switch** for a problem.
 - **Always pay attention to your indentation.**
 - Also, as a good programming habit, make sure you make the **most common case** the first one.
-

Loops

- Loops are repetitive blocks.
- We need them almost all the time.
- Here is a problem for you:

You want to find the average grade in a class. How can you write this program?

Reasoning

- To write any program you need to know the solution.
 - Calculating an average is simple: you sum up all the values and divide the result by the number of values.
 - To make the program calculate the average using the solution above, we need to write it step by step using the constructs that we know.
-

Reasoning

- We can ask for input for each student. The input we get can be added at each step to another variable.
 - Once all the grades are entered, we divide the sum (1) to the number of students (2).
 - We know how to get input from the user. So, we will use **scanf**.
 - We can use mathematical operators to add numbers and divide the result to the total number of students.
-

Reasoning

- The **scanf** function needs a variable to store the input value. We can call this variable **temp** and assuming grades are integers, we can use type **int**.

```
int temp = 0;
```

- The expression above initializes the variable to 0. You should always initialize your variables. In this case, the value 0 will be overwritten when the user enters a new value, but it is OK. We play it safe.
-

Reasoning

- Notice that we don't need to keep every user input. We just need to add them one after another.
 - Therefore, we need another integer. Let's call it `sum`.

```
int sum = 0; // (1)
```
 - This time the initialization makes sense because we will be adding the **temp** value to **sum**.
-

Reasoning

- The average is calculated by dividing the **sum**, by the number of values.
 - Since we don't know any loop constructs yet, the number of students (2) will be hard-coded to the program.
 - For the sake of brevity, let's keep it at three.
 - Without loops, I'll be using ancient wisdom: copy & paste.
-

```
1  #include <stdio.h>
2
3  int main() {
4      int temp = 0;
5      int sum  = 0;
6
7      printf("Enter the grade for student: ");
8      scanf("%d", &temp);
9      sum += temp;
10
11     printf("Enter the grade for student: ");
12     scanf("%d", &temp);
13     sum += temp;
14
15     printf("Enter the grade for student: ");
16     scanf("%d", &temp);
17     sum += temp;
18
19     printf("The average value is: %f\n", (float)sum/3.0);
20     return 0;
21 }
22
```

"C:\Users\Kaya O-uz\Desktop\average.exe"

Enter the grade for student: 60
Enter the grade for student: 70
Enter the grade for student: 81
The average value is: 70.333333

Process returned 0 (0x0) execution time : 6.867 s
Press any key to continue.

Problems

- We have used copy & paste.
 - If the number of students change, so does the code.
 - We have an integer constant in the average calculation (3.0). If the number of students change, we should not forget to change it as well.
 - How can loops help?
 - First, identify the repetition, then use the loop construct.
-

```
1  #include <stdio.h>
2
3  int main() {
4      int temp = 0;
5      int sum  = 0;
6
7      printf("Enter the grade for student: ");
8      scanf("%d", &temp);
9      sum += temp;
10
11     printf("Enter the grade for student: ");
12     scanf("%d", &temp);
13     sum += temp;
14
15     printf("Enter the grade for student: ");
16     scanf("%d", &temp);
17     sum += temp;
18
19     printf("The average value is: %f\n", (float)sum/3.0);
20     return 0;
21 }
22
```


Loops

- A loop has three parts:
 - a. The initial value(s)
 - b. The step
 - c. The stopping condition
 - The initial value and the step may not always be there.
 - The stopping condition is crucial. If this condition is never met, the loop does not stop. We call such loops as **infinite loops**. It makes your computer hang (=this is bad).
-

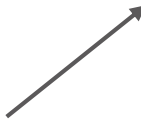
The increment operator

- The step is almost always **one** step at a time. So, we usually use the increment operator: ++
 - Remember that ++ operator meant $x = x + 1$;
 - There are two kinds of it: post and pre.
 - You know that
`printf("%d\n", x + 1);`
will print the value of $x + 1$.
 - Consider the following case:
-

Why?

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5      printf("%d\n", i++);
6      return 0;
7  }
```

```
Kaya-iMac:Desktop koguz$ gcc post.c -o post
Kaya-iMac:Desktop koguz$ ./post
0
Kaya-iMac:Desktop koguz$
```



```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5      printf("%d\n", i++);
6      printf("%d\n", i);
7      return 0;
8  }
```

```
Kaya-iMac:Desktop koguz$ gcc post.c -o post
Kaya-iMac:Desktop koguz$ ./post
0
Kaya-iMac:Desktop koguz$ gcc post.c -o post
Kaya-iMac:Desktop koguz$ ./post
0
1
Kaya-iMac:Desktop koguz$
```



variable++ is post-increment.

- The post-increment returns the original value. However, the next time the variable is used, the incremented value is used.
 - The pre-increment returns the incremented value. The next time, the value is still incremented.
 - The same goes for the decrement operator.
 - So, if you do
`x = i++;`
the x variable has the value of i before increment.
-

Examples:

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5      printf("%d\n", ++i);
6      printf("%d\n", i);
7      return 0;
8  }
```

```
Kaya-iMac:Desktop koguz$ gcc pre.c -o pre
Kaya-iMac:Desktop koguz$ ./pre
1
1
Kaya-iMac:Desktop koguz$
```

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5      printf("%d\n", i++);
6      printf("%d\n", i);
7      printf("%d\n", --i);
8      return 0;
9  }
```

```
Kaya-iMac:Desktop koguz$ ./pre
0
1
0
Kaya-iMac:Desktop koguz$
```

while

- The **while** loop is a simple loop where you only need a stopping condition.
- The loop runs as long as the condition is non-zero (true).

```
while (expression) {  
    statements;  
}
```

while

- Let's try a simple example:

```
1  #include <stdio.h>
2
3  int main() {
4      int counter = 0;
5      while(counter < 10) {
6          printf("SE 115\n");
7          counter++;
8      }
9      return 0;
10 }
11
```

initial value

stopping condition

step

Notice the three loop parts as well :)

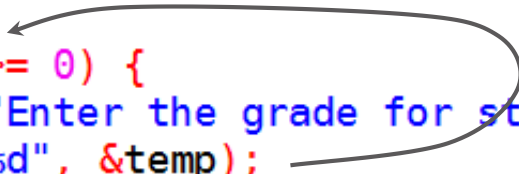
Reasoning

- Now that we know how to write a loop, let's try to use it.
 - We can go ahead and use the code above and set the stopping condition to (counter < 3).
 - However, let's generalize it more and make it work with any number of students!
 - To do so, we need to change the stopping condition to **something** that will **indicate** the end of students.
-

The Stopping Condition

- The grades are all positive, so if the user enters a grade less than zero, we can stop.
 - This time, we need to know how many students the user will enter.
 - It is easy; at the end of every loop, we can increment a counter.
 - **If** the user enters a negative number, the value entered will **not** be added to the sum and the counter will **not** be incremented.
 - Let me try again.
-

```
1  #include <stdio.h>
2
3  int main() {
4      int temp    = 0;
5      int sum      = 0;
6      int counter  = 0;
7      float avg    = 0.0f;
8
9      while(temp >= 0) {
10         printf("Enter the grade for student (negative to end): ");
11         scanf("%d", &temp);
12         if(temp >= 0) {
13             sum += temp;
14             counter++;
15         }
16     }
17
18     avg = (float)sum / (float)counter;
19     printf("The average value is: %f\n", avg);
20
21     return 0;
22 }
23
```



Remarks

- We check the **temp** variable twice. Can we do it at one take?
 - The variable called counter is of type **int**. While calculating **avg**, we cast it to a **float**. Can we do it without a cast?
 - These are your **homework** assignments (not to be graded).
-

while

- With the introduction of **while**, you have learned about the three parts of a loop.
 - You know that you can loop a number of times, or let the user decide it.
 - **Homework:** re-write the average program and ask the user for the number of students first and loop that many times.
Enter number of students: 10
Enter grade for student 1: ...
Enter grade for student 2: ...
-

Loops

- So, you know about loops now. I'll introduce two more: **do...while** and **for** loops.
 - You'll notice that they are also similar.
 - Remember; the starting conditions, the step and the stopping condition.
 - You can tweak these according to your needs.
-

do...while

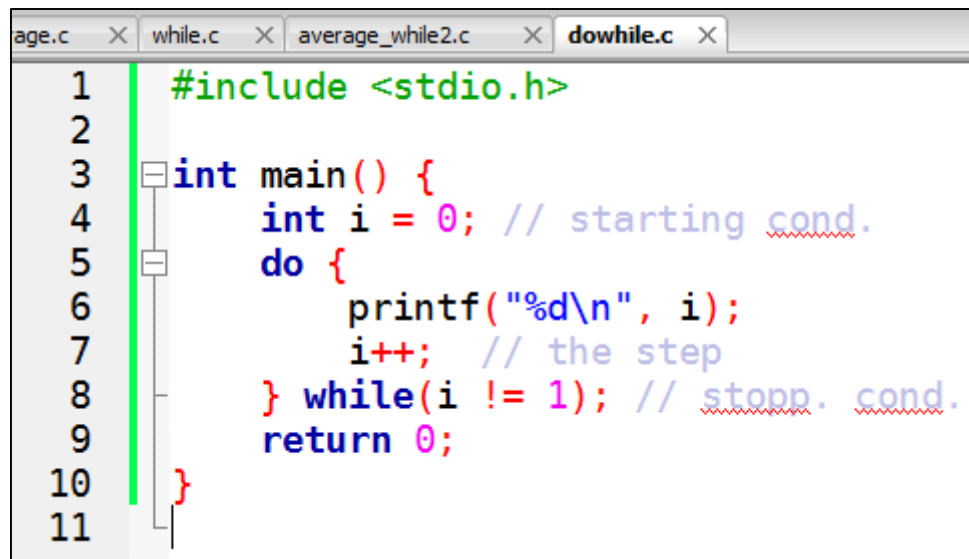
- The **do...while** loop has a structure as follows:

```
do {  
    statements;  
} while (expression) ;
```

- Notice that the statements are run at least once (the flow). Similar to **while**, the loop continues until expression returns zero.
 - Notice the semi-colon at the end of while.
-

do...while

- Running the body at least once is the characteristic of **do...while** loops.
- If you run the code below, you'll notice that the **printf** is executed once.



```
age.c x while.c x average_while2.c x dowhile.c x
1  #include <stdio.h>
2
3  int main() {
4      int i = 0; // starting cond.
5      do {
6          printf("%d\n", i);
7          i++; // the step
8      } while(i != 1); // stopp. cond.
9      return 0;
10 }
11
```

for

- The **for** loop is one of the most famous loops. It is also used quite often.
- The three parts of a loop exist in the definition of the loop:

```
for(initialize; stopping condition; step) {  
    statements;  
}
```

for

- Here is an example.
- Notice that I've initialized the variable `i` to 23 during declaration.
- However, the for loop initializes it to 0, the stopping condition is `i<10` and the step is `i++`.

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 23;
5      for(i=0; i<10; i++) {
6          printf("%d\n", i);
7      }
8      return 0;
9  }
10
```

for

- I have already asked you to write the average program again by asking the user the number of students to enter.
 - This time, instead of **while**, use **for**.
-

for

- You don't have to write anything at the declaration of the for loop.
- For example, if the loop variable is already initialized, skip the first part.

```
int i = 0;  
for( ; i<10; i++) { }
```

- If you don't want to check the stopping condition (are you sure?) then skip the part in the center as well.

```
for( ; ; i++) { }
```

for

- If you will increment the value within the loop (according to a condition perhaps), then skip the last one as well.

```
for ( ; ; ) { }
```

- Actually, these conditions are rare.
 - It is because you want to increase code readability.
 - The **for** loop provides a quick glance to all of the three loop parts.
-

for

- One more thing: you can use comma (,) within these three loop parts. So, you can use two variables within the loop.

```
int i, j;
```

```
for(i=0, j=10; i<10; i++, j--) { }
```

- Yes, you can use a negative step. Why not write a program that will print the numbers starting from a value to zero. What will be the stopping condition? How can you do it while you are incrementing the step?
-

Why not increment more than one?

- What if you want to increment by 2, rather than by 1?

```
int i;
```

```
for (i=0; i<100; i+=2) { ... }
```

More about loops

- I assume that you got a sense of using the loops.
 - There are a few things you can do with loops, such as **nesting** them (we like nesting, don't we?)
 - That means using a loop, inside a loop.
-

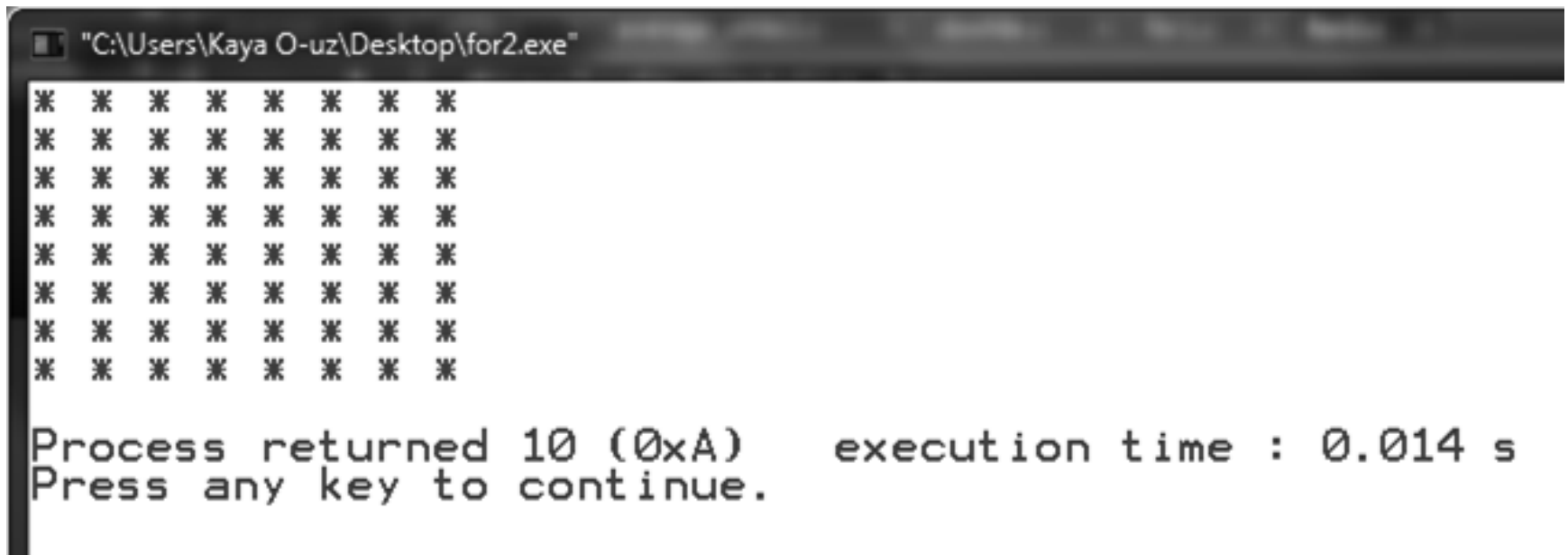
Nested Loops

```
// let's say this loop runs n times
for( ... ; ... ; ... ) {
    /* some statements */
    // and this one runs m times
    for( ... ; ... ; ... ) {
        // the statements here
        // n*m times
    }
}

// notice the indentation.
// you can easily see the blocks.
```

Nested Loop Problem

- Let's try some graphics (not 3D).
- Why not draw a square using a character?



```
"C:\Users\Kaya O-uz\Desktop\for2.exe"  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
  
Process returned 10 (0xA)    execution time : 0.014 s  
Press any key to continue.
```

The screenshot shows a Windows command prompt window titled "C:\Users\Kaya O-uz\Desktop\for2.exe". The window displays a square of asterisks, which is 8 rows high and 8 columns wide. Below the square, the text "Process returned 10 (0xA)" and "execution time : 0.014 s" is visible. At the bottom, the prompt "Press any key to continue." is shown.

Reasoning

- The square is a simple ASCII art.
 - Since it is a square there are equal number of columns and rows.
 - The command line window outputs text from left to right.
 - On every line, we need to print **n** number of characters.
 - The “printing of n number of characters” will be done **n** times. So, there will be **n^2** characters on the screen.
-

Reasoning

- Remember that we need to find the repetitions to use with the loops.
 - The first repetition is the single line.
 - The second repetition is printing a number of lines.
 - The problem here is which loop will be the inner loop?
 - Go back a few more slides and look at the nested loop template.
-

Step 1

- Let's try hard-coding first.
- This will print the 8x8 square.
- The repetition in the code below are the stars (*). Let's put them to the loop as well.
- Notice that the new line `\n` does not repeat.

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5      int j = 0;
6
7      for(i=0;i<8;i++) {
8          printf("* * * * * \n");
9      }
10 }
11
```

Step 2

- The inner loop (with variable j) runs 64 times and prints “* “ on screen. Once the 8 stars are complete, we print \n to get a new line.

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5      int j = 0;
6
7      for(i=0;i<8;i++) {
8          for(j=0;j<8;j++) {
9              printf("* ");
10         }
11         printf("\n");
12     }
13 }
14
```

Homework:

This program has a constant integer value: 8. Re-write it so that the user enters the size of the square.

Nesting other loops

- You can nest a while loop inside another while loop.
 - You can nest a while loop inside another for loop.
 - You can nest as deep as you require, with any loop you need.
 - Just watch out the variable you use. If you use variable `i` for the outmost loop, use another variable for the inner loop.
-

break

- Remember the keyword **break** from **switch** we have discussed.
 - If you use **break** within a loop, you break out of it.
 - I want to use this opportunity to demonstrate the infinite loop. It is fun to **break** from the infinite loop.
-

Infinite Loop

- If you remove the **break**, your screen will be filled with “infinite loop”
- Instead of `for(;;)`, you can try `while(1)`

```
1  #include <stdio.h>
2
3  int main() {
4      for(;;) {
5          printf("infinite loop!\n");
6          break;
7      }
8      printf("infinite loop breaks!\n");
9      return 0;
10 }
11
```


Breaking Infinite Nested Loops

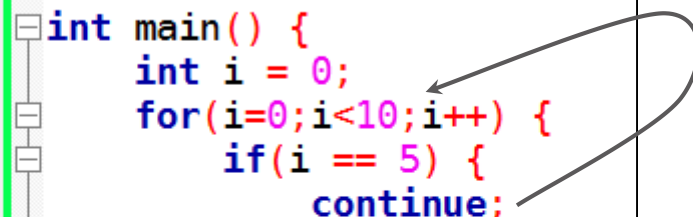
- This demonstrates that you **break** out of the loop you are in.

```
1  #include <stdio.h>
2
3  int main() {
4      for(;;) {
5          for(;;) {
6              printf("inside inner loop\n");
7              break;
8          }
9          printf("inner loop breaks\n");
10         printf("infinite loop!\n");
11         break;
12     }
13     printf("infinite loop breaks!\n");
14     return 0;
15 }
16
```

continue

- The **continue** keyword is used to skip the remaining statements in the loop body.
- You can use it with all of them.
- The code below won't print 5.

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5      for(i=0;i<10;i++) {
6          if(i == 5) {
7              continue;
8          }
9          printf("%d\n", i);
10     }
11     return 0;
12 }
13
```



Game Programming

- You have acquired skills to write your first computer game.
 - It is a very old game and can be easily programmed using **conditionals** and **loops**.
 - The program will store a random integer, and the user (player) will try to guess the number.
 - At each guess, the program will give a hint, such as “too high!” or “too low!”.
 - You can use **rand()** function to get a random number (I’ll demonstrate it).
 - When the player guesses correctly, the program will print the number of times he failed.
-

```
koguz@mate: ~
File Edit View Search Terminal Help
koguz@mate:~$ nano r.c
koguz@mate:~$ gcc r.c -o r
koguz@mate:~$ ./r
383
386
277
415
293
335
386
492
149
421
koguz@mate:~$
```

rand() returns a random integer. We use modulus to limit it.

```
koguz@mate: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: r.c

#include <stdio.h>

int main() {
    int i=0;
    for(i=0;i<10;i++) {
        printf("%d\n", rand()%500);
    }
    return 0;
}

[ Read 9 lines ]
^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text ^T To Spell
```

Here is a sample output

```
ce350@debian7:~$ gcc guess.c -o g
ce350@debian7:~$ ./g
Guess my number: 250
Too low
Guess my number: 375
Too low
Guess my number: 430
Too high
Guess my number: 400
Too high
Guess my number: 387
Too high
Guess my number: 381
Too low
Guess my number: 384
Too high
Guess my number: 382
Too low
Guess my number: 383
Congratulations!
Number of attempts: 9
ce350@debian7:~$
ce350@debian7:~$
ce350@debian7:~$ _
```

Tips

- You have seen a screenshot of a sample output.
 - Role-play as the program, and try to think about what it does at every input.
 - Once you have a clear idea of the steps to take, merely use the tools in your possession to write the code.
-

Bonus

- Also, write another program that will try to guess the number you hold.
 - The program will ask you for an upper limit.
 - Then, it will start guessing and you will reply as high or low (0 or 1).
 - The program must be clever to keep the number of guesses as few as it can.
-

Quest

- In a **for** loop of that runs 30 times, print the output of **rand()%100**. It will give you 30 random numbers, up to 100.
 - Run the program again.
 - Anything weird?
 - Run again.
 - Still?
 - Why does it happen? How can it be prevented?
 - How can you design a program that creates random values?
-

Functions

- You are familiar with functions
 - We wrote a **main** function
 - We called (used) functions like **printf** and **scanf**.
 - You are also familiar with their properties
 - They have names as identifiers
 - They have parameters
 - They have types and they return that type
 - But, why do we need them?
-

Why?

- The official reason is to divide the problem into smaller pieces that are easier to maintain: *divide and conquer*
 - Functions are also the building blocks of more complex programs.
 - Think about **printf** and **scanf**. We use them to create programs for different purposes.
 - It is also a means of *abstraction*.
-

Abstraction?

- Think about the light switch.
- You hit a button and it toggles the lights.
- Your input (parameter) is the button position.
- You don't know anything about the electrical background such as the cables and the electrical power.
- As a result the lights are turned on or off.
- It returns nothing (hopefully).



Abstraction

- The light switch might have other abstractions which even the electrician may not know about.
 - As a result the light switch has a purpose with clear boundaries and an interface.
 - The other important abstraction in programming is data abstraction. It is about creating your own types from other types. (We'll talk about data abstraction two weeks later.)
-

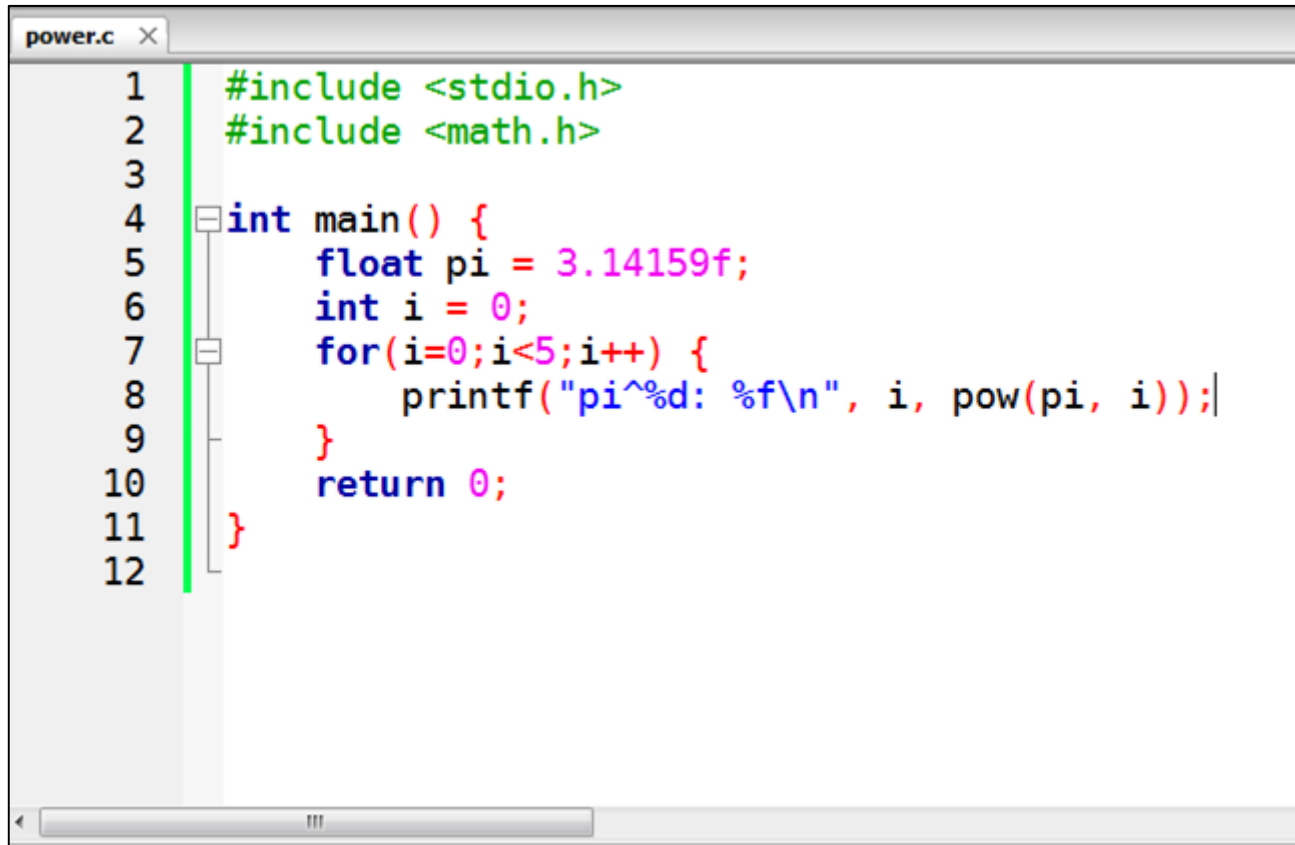
Why?

- Other than abstraction, it is also possible to create a function from a list of statements that are part of a job.
 - For example, a **menu** function can print a list of functions available in the program.
-

Functions

- Mathematical functions such as **power** (exponentiation), **square root** and **factorial** are all good candidates for C functions.
 - However, most of them are already in `math.h` library file.
 - Before writing functions of our own, let's take a look at one of them.
 - The **pow** function takes two parameters of type **double** and returns a **double** value.
-

The pow function.



```
1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      float pi = 3.14159f;
6      int i = 0;
7      for(i=0; i<5; i++) {
8          printf("pi^%d: %f\n", i, pow(pi, i));
9      }
10     return 0;
11 }
12
```

On Linux and Mac, you may need to compile this file as follows:

gcc power.c -o power -lm

The -lm parameter links the mathematical functions. You don't need it with Codeblocks.

Type of the function

- The type of the function defines the **return** type.
 - Remember that with **printf** we can use the functions as parameters.
 - If the function **returns** an integer, then we can use %d to print the result, just like printing the value of an integer.
-

The void type

- A new type for functions is **void**.
- The **void** type is used when the function does not return anything.

The parameters

- Although not required, most functions have parameters.
 - When you think of it, it makes sense: we will probably want to work on some *values*. It would be useful to be able to *transfer* (pass) them to the function.
 - The **printf** function requires a parameter for output, and a number of parameters for format specifiers.
-

Body of a function

- The body is where the action takes place.
 - It is written between curly braces and contains a number of statements.
 - The last line is the **return** line. Any statement below **return** line will be not be executed.
 - If the type is **void**, just type **return**.
 - You can define variables in the body of the function. These variables are **local** to the function.
-

Locals?

- The variables are valid inside the curly braces they are defined in. They can not be accessed outside. This is called **scope**.
 - Official definition: The scope of a name is the part of the program within which the name can be used.
 - The rules are very simple, but before the rules, let's see an example.
-

*scope.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

power.c X *scope.c X

```
1  #include <stdio.h>
2
3  int main() { // the scope of main begins
4      int a = 0;
5      int i = 0;
6      for(i=0;i<10;i++) { // scope of for loop
7          int k = 10; // k for kaya
8          k++;
9          printf("%d: %d\n", a, k);
10     }
11     // error: 'k' undeclared
12     printf("value of k is %d\n", k);
13
14 } // end of the scope for main
15
```

Variables a and i are declared inside main, so they are accessible in the for loop as well.

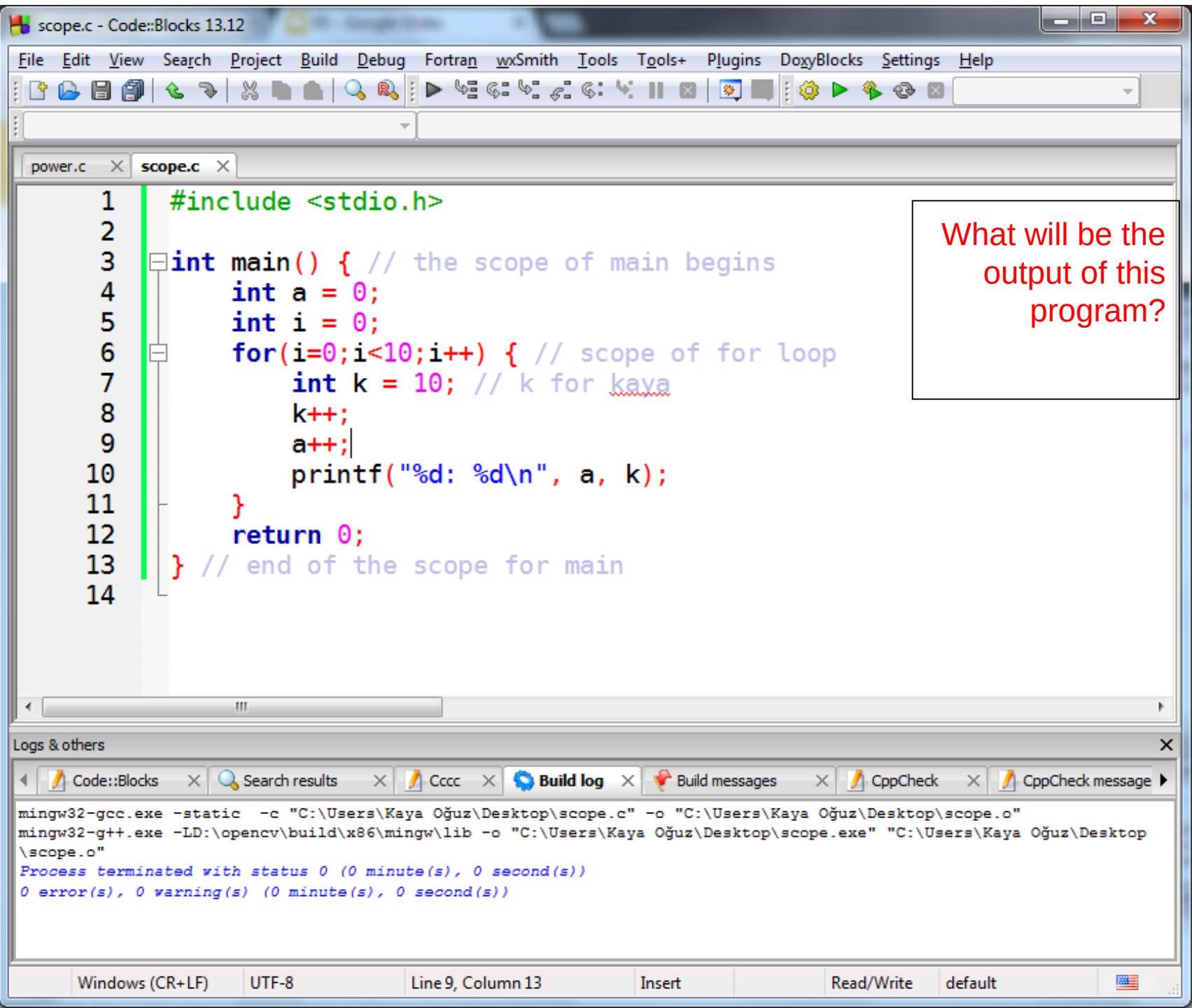
However, k is defined in the scope of the for loop. It can not be accessed from outside of the loop.

Logs & others

Code::Blocks X Search results X Cccc X Build log X Build messages X CppCheck X CppCheck mess

File	Line	Message
C:\Users\Kaya ...		In function 'main':
C:\Users\Kaya ...	12	error: 'k' undeclared (first use in this function)
C:\Users\Kaya ...	12	note: each undeclared identifier is reported only once for each function it appears in
== Build failed: 1 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ==		

C:\U Windows (CR+LF) UTF-8 Line 11, Column 29 Insert Modified Read/Write default



Locals

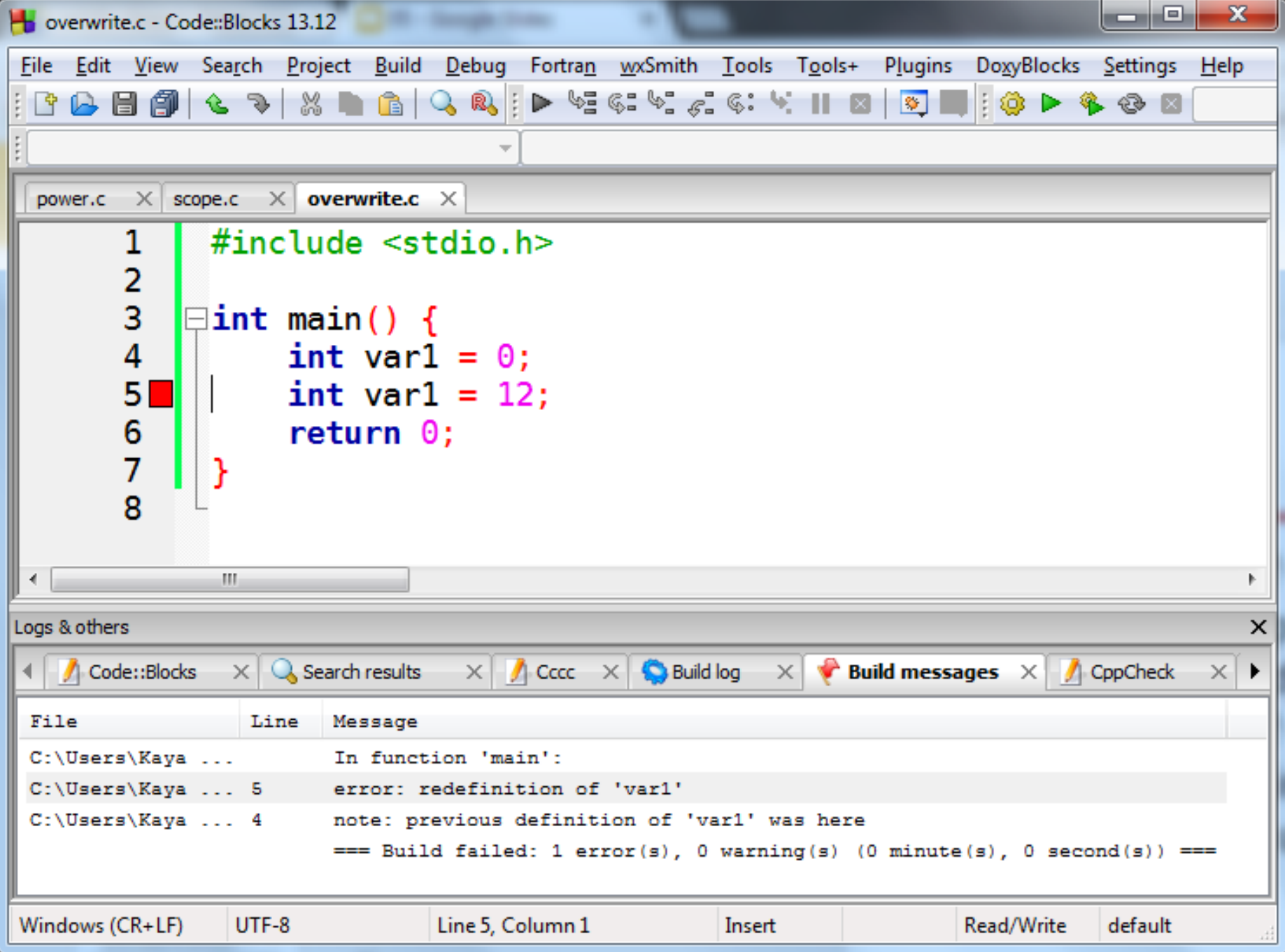
- Since we are writing functions between curly braces, any variables declared in the function will be available only in that function.
 - You can declare variables outside the main function. They are called **global variables**. They can be accessed from any function.
 - However, it is not a good programming practice to use globals. In more complicated programs, you can not find out the source of a bug if every function gets access to the variable.
-

```
1  #include <stdio.h>
2
3  int globalvariable = 12;
4
5  int main() {
6      printf("%d\n", globalvariable);
7      for(;globalvariable<15;globalvariable++) {
8          printf("%d\n", globalvariable);
9      }
10     return 0;
11 }
```

The **globalvariable** is global and can be accessed from any function.

Overwriting

- What happens if we declare a variable twice?
 - The compiler tells you that the variable is already declared.
 - What happens if we declare a variable with the same name **but** in another scope? Which variable will be valid?
-



power.c X scope.c X overwrite.c X

```
1  #include <stdio.h>
2
3  int main() {
4      int var1 = 0;
5      int i;
6      for(i=0;i<10;i++) {
7          int var1 = 12;
8          printf("%d\n", var1);
9      }
10     return 0;
11 }
12
```

The variable var1
is redefined in the
scope of for loop.
The printf will
print the value of
12 in that line.

Logs & others

Code::Blocks X Search results X Cccc X Build log X Build messages X CppCheck X

```
\overwrite.o"
mingw32-g++.exe -LD:\opencv\build\x86_mingw\lib -o "C:\Users\Kaya Oğuz\Desktop\overwrite.exe" "C:
\Users\Kaya Oğuz\Desktop\overwrite.o"
Process terminated with status 0 (0 minute(s), 0 second(s))
0 error(s), 0 warning(s) (0 minute(s), 0 second(s))
```

overwrite.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

power.c x scope.c x overwrite.c x

```
1  #include <stdio.h>
2
3  int main() {
4      int var1 = 3;
5      int i;
6      for(i=0; i<10; i++) {
7          int var1 = 12;
8          printf("%d\n", var1);
9      }
10     printf("%d\n", var1);
11     return 0;
12 }
13
```

Now what happens here?
What will be the output on line 10?

Logs & others

Code::Blocks x Search results x Cccc x Build log x Build messages x CppCheck x

Checking for existence: C:\Users\Kaya Oğuz\Desktop\overwrite.exe
Executing: C:\Program Files (x86)\CodeBlocks\cb_console_runner.exe "C:\Users\Kaya Oğuz\Desktop\overwrite.exe" (in C:\Users\Kaya Oğuz\Desktop)
Process terminated with status 0 (0 minute(s), 1 second(s))

Windows (CR+LF) UTF-8 Line 10, Column 26 Insert Read/Write default

Function Definition

- You now know all the prerequisites.
 - Then, it is time to write our own functions.
 - Remember the basics:
 - The function has a type;
 - The function has a name (a valid identifier);
 - The function has parameters (zero or more);
 - The function has a body with a list of statements inside the curly braces;
 - The function can have variables defined in the body, called the locals;
 - If the function will not return anything, we can use the type void;
-

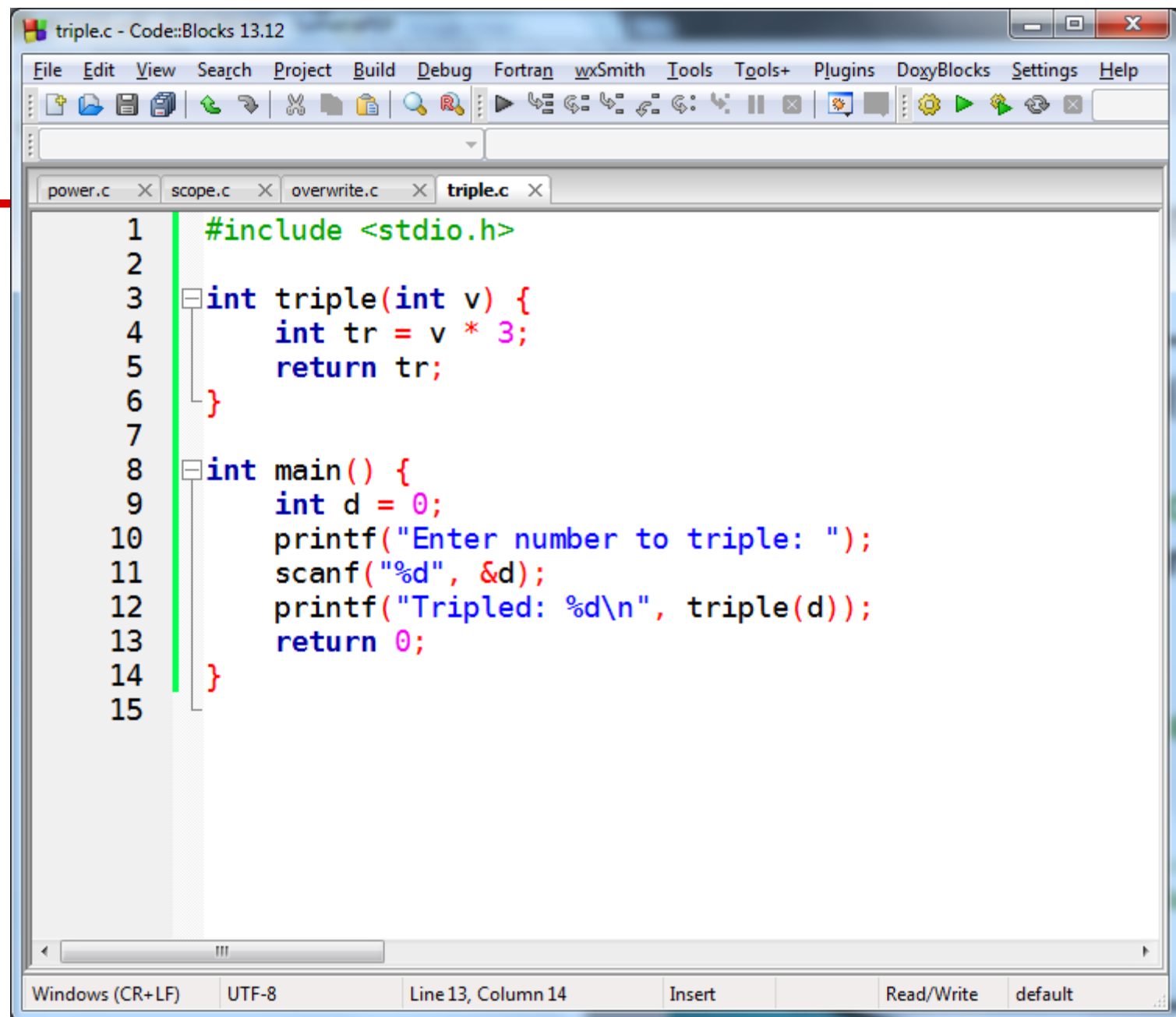
Syntax

```
type function_name (type p1, type p2) {  
    declarations  
    statements  
    return expression;  
}
```

```
type function_name () {  
    declarations  
    statements  
    return expression;  
}
```

Tripling an integer

- Suppose we want to triple an integer.
 - Tripling is simply multiplication by three.
 - This is a very simple function.
 - Let's think about the return type.
 - Let's think of a valid name.
 - Do we need any parameters?
 - What do we need in the function?
-



triple.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

power.c x scope.c x overwrite.c x triple.c x

```
1  #include <stdio.h>
2
3  int triple(int v) {
4      int tr = v * 3;
5      return tr;
6  }
7
8  int main() {
9      int d = 0;
10     printf("Enter number to triple: ");
11     scanf("%d", &d);
12     printf("Tripled: %d\n", triple(d));
13     return 0;
14 }
15
```

Windows (CR+LF) UTF-8 Line 13, Column 14 Insert Read/Write default

Step by Step

- Remember the flow of a program.
 - Since we start from the beginning, the function must be declared before we can use it.
 - In main, we use variable **d** to get input from user. We send the *value* of variable **d** to the function triple, where it is saved in variable **v** as a parameter (and as a **local**).
-

Step by Step

- We work on the *value* using the variable **v** and **return** the result.
 - The result is returned and used as the second parameter in the printf function.
 - The value is printed on the screen.
-

Calling a function

- To call a function, you write the name of the function and provide the required argument values (to be assigned to the corresponding parameters within the function definition).
 - The flow of the program changes into the function.
 - When the function **returns**, the flow is back on the calling function, to the line it was called.
 - A function can call other functions (it can even call itself, but shhhh!)
-

Calling Functions

1. It all starts from main.

int triple

float half

int main

Let's say we have defined two functions. Every program has main. Let's take a look at the flow.

```
int main() {  
    int a = 3;  
    int t = triple(a);  
    printf("%f\n", half(t));  
    return 0;  
}
```

2. The flow passes to this function

```
int triple(int d)  
{  
    /* ... */  
    return d;  
}
```

3. and returns back

4. now to this function

```
float half(int d)  
{  
    /* ... */  
    return d;  
}
```

5. and returns where it was called

More on Functions

- There are more things to consider about functions.
- It is better to demonstrate them.

triple_2.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main() : int

triple_2.c x

```
1  #include <stdio.h>
2
3  int triple(int d) {
4      return d * 3;
5  }
6
7  int main() {
8      int t = 0;
9      printf("Enter value to triple: ");
10     scanf("%d", &t);
11     t = triple(t);
12     printf("Tripled: %d\n", t);
13     return 0;
14 }
15
```

Windows (CR+LF) UTF-8 Line 10, Column 21 Insert Read/Write default

Improving triple function

- We had a redundant variable in the previous version. We can return the value immediately.
 - The assignment on line 11 is important.
 - Remember that the = sign is assignment, and sets the value on the right to the variable on the left.
 - The value of variable **t** is sent to the triple function and the returned value is assigned as the value of **t**.
 - Notice that the return type of triple function is the same of the variable **t**.
-

menu.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> menu() : void

triple_2.c x menu.c x

```
1  #include <stdio.h>
2
3  void menu() {
4      printf("****MENU****\n\n");
5      printf("1. Say \"hello\"\n\n");
6      printf("2. Absolute value\n");
7      printf("3. Sum between numbers\n");
8      printf("4. Quit\n\n");
9  }
10
11 int main() {
12     int choice = 0;
13     menu(); // call the menu
14     scanf("%d", &choice);
15     switch(choice) {
16         /* ... */
17     }
18     return 0;
19 }
20
```

Windows (CR+LF) UTF-8 Line 6, Column 35 Insert Read/Write default

void

- So, here is a sample menu function with type void.
 - Notice that I have not used a **return** line.
 - It prints the menu when required.
 - **Homework:** write a **function** for the menu option 1. It will only print hello on screen.
-

Absolute

- Let's write the absolute function.
 - The absolute function returns the absolute value of an integer.
 - So we need an integer as a parameter.
 - If the value is less than zero, we can return the value multiplied by -1.
-

menu.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global>

triple_2.c x menu.c x

```
4      printf("****MENU****\n\n");
5      printf("1. Say \"hello\"\n");
6      printf("2. Absolute value\n");
7      printf("3. Sum between numbers\n");
8      printf("4. Quit\n\n");
9  }
10
11  int absolute(int a) {
12      if(a < 0) {
13          return a * (-1);
14      } else return a;
15  }
16
17  int main() {
18      int choice = 0;
19      menu(); // call the menu
20      scanf("%d", &choice);
21      switch(choice) {
22          /* ... */
23      }
24      return 0;
```

Windows (CR+LF) UTF-8 Line 10, Column 1 Insert Read/Write default

Sum

- This function requires two integer parameters.
 - The numbers between these numbers are summed and returned.
-

menu.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global>

summation(int a, int b) : int

triple_2.c x menu.c x

12 if(a < 0) {
13 return a * (-1);
14 } else return a;
15 }
16
17 int summation(int a, int b) {
18 int sum = 0;
19 int i = 0;
20 for(i=a;i<b;i++) {
21 sum += i;
22 }
23 return sum;
24 }
25
26 int main() {
27 int choice = 0;
28 menu(); // call the menu
29 scanf("%d", &choice);
30 switch(choice) {
31 /* ... */
32 }

Windows (CR+LF) UTF-8 Line 23, Column 16 Insert Read/Write default

Sum

- There is something to consider in the summation function.
 - If **a** is greater than **b**, then the loop does not run (the stopping condition will be true at the first step).
 - Let it be your homework.
-

Function Prototypes

- Remember the flow of the program.
 - You can not use the function before defining them.
 - But there is a way to declare without the body.
 - You don't have to write the variable names in function prototypes.
-

*prototype.c - Code::Blocks 13.12

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> summation(int a, int b) : int

triple_2.c x menu.c x *prototype.c x

```
1  #include <stdio.h>
2
3  void menu();
4  int absolute(int);
5  int summation(int a, int b);
6
7  int main() {
8      /* main */
9      return 0;
10 }
11
12 void menu() {
13     printf("1. Say \"hello\\\"\\n");
14 }
15
16 int absolute(int a) {
17     /* */
18 }
19
20 int summation(int a, int b) {
21     /* */
22 }
23
```

Windows (CR+LF) UTF-8 Line 21, Column 11 Insert Modified Read/Write default

Scope

- Remember that if a variable is defined in a **scope**, it is only valid inside that scope.
 - If that scope is a function, it lives only in that function.
 - Global variables are defined outside functions (for example just after **includes**.)
-



Start here x local_global.c x

```
1  #include <stdio.h>
2
3  int a_global_variable = 12;
4  int scope_sum(int n);
5
6  int main() {
7      int v = 8;
8      printf("value of v: %d\n", v);
9      printf("scope_sum: %d\n", scope_sum(v));
10     printf("v after function: %d\n", v);
11     return 0;
12 }
13
14 int scope_sum(int n) {
15     n = n + 2;
16     return n;
17 }
18
```

"C:\Users\Kaya O-uz\Desktop\local_global.exe"

value of v: 8
scope_sum: 10
v after function: 8

Process returned 20 (0x14) execution time : 0.013 s
Press any key to continue.

Why?

- Remember, I always emphasize “the **value** of a variable”.
 - Here, we send the **value** of variable *v* to the function. We are not working **on** the variable, we are working **with the value** of the variable.
 - So, the variable is safe inside main.
 - This way of passing parameters to functions is called: CALL BY VALUE.
-

A Global Example

- You may have noticed the global variable.
- What happens if we change it inside the **scope_sum** function?



Start here x local_global.c x

```
1  #include <stdio.h>
2
3  int a_global_variable = 12;
4  int scope_sum(int n); // a global
5
6  int main() {
7      int v = 8;
8      printf("value of v: %d\n", v);
9      printf("scope_sum: %d\n", scope_sum(v));
10     printf("v after function: %d\n", v);
11     printf("global: %d\n", a_global_variable);
12     return 0;
13 }
14
15 int scope_sum(int n) {
16     n = n + 2;
17     a_global_variable = 16;
18     return n;
19 }
20
```

Storage Classes

- A variable has a name, as an identifier.
 - It has a type.
 - It has a memory space.
 - They also have a storage class.
 - auto, extern, register, static
 - It determines storage **duration**, **scope** and **linkage** of the variable.
 - Linkage is about the names and the entities (variables) during the running of the program.
-

auto

- Variables declared within function bodies are auto by default.
 - So far, all the variables had the auto storage class.
 - Since it is default, we do not need to write it.
-

extern

- Informally, look for the variable elsewhere, either in this file, or in some other file.
 - External variables are defined outside of any function, and are thus potentially available to many functions.
 - Functions are always external.
 - External variables are also useful because of their greater scope and lifetime.
-

extern sample

```
p1.c x file1.h x
1  #include <stdio.h>
2  #include "file1.h"
3
4  int a=1, b=2, c=3;
5
6  int f();
7
8  int main() {
9      printf("%d %d %d\n", a, b, c);
10     printf("%d\n", f());
11     printf("%d %d %d\n", a, b, c);
12     return 0;
13 }
14
```

```
file1.h x
1  int f() {
2      extern int a;
3      a = 4;
4      return a;
5  }
6
```

```
"C:\Users\Kaya O-uz\Desktop\p1.exe"
1 2 3
4
4 2 3

Process returned 0 (0x0)
Press any key to continue.
```

register

- A `register` declaration advises the compiler that the variable in question will be heavily used.
 - The idea is that `register` variables are to be placed in machine registers, which may result in smaller and faster programs.
 - But, compilers are free to ignore the advice.
 - The `register` declaration can only be applied to automatic variables and to formal parameters of a function.
-

static

- The `static` declaration applied to internal variables are local to a particular function and they **remain in existence** rather than coming and going each time the function is activated.
 - It will be clearer with a demonstration.
-

```
1  #include <stdio.h>
2
3  void f() {
4      int c = 12;
5      printf("%d\n", ++c);
6      return;
7  }
8
9  int main() {
10     int i = 0;
11     for(i=0; i<5; i++) {
12         f();
13     }
14     return 0;
15 }
16
```

"C:\Users\Kaya O-uz\Desktop\nonstatic.exe"

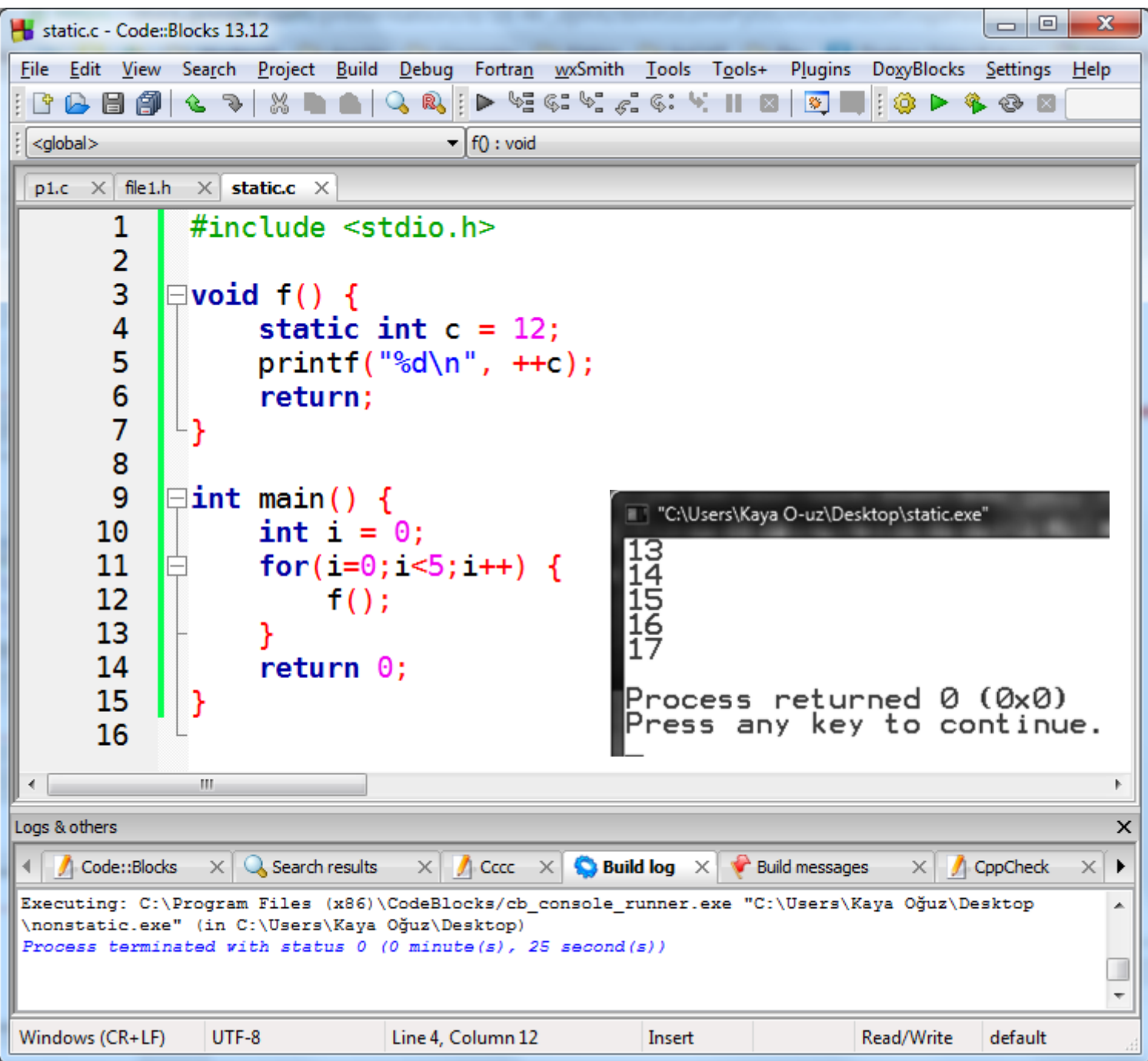
13
13
13
13
13

Process returned 0 (0x0)
Press any key to continue.

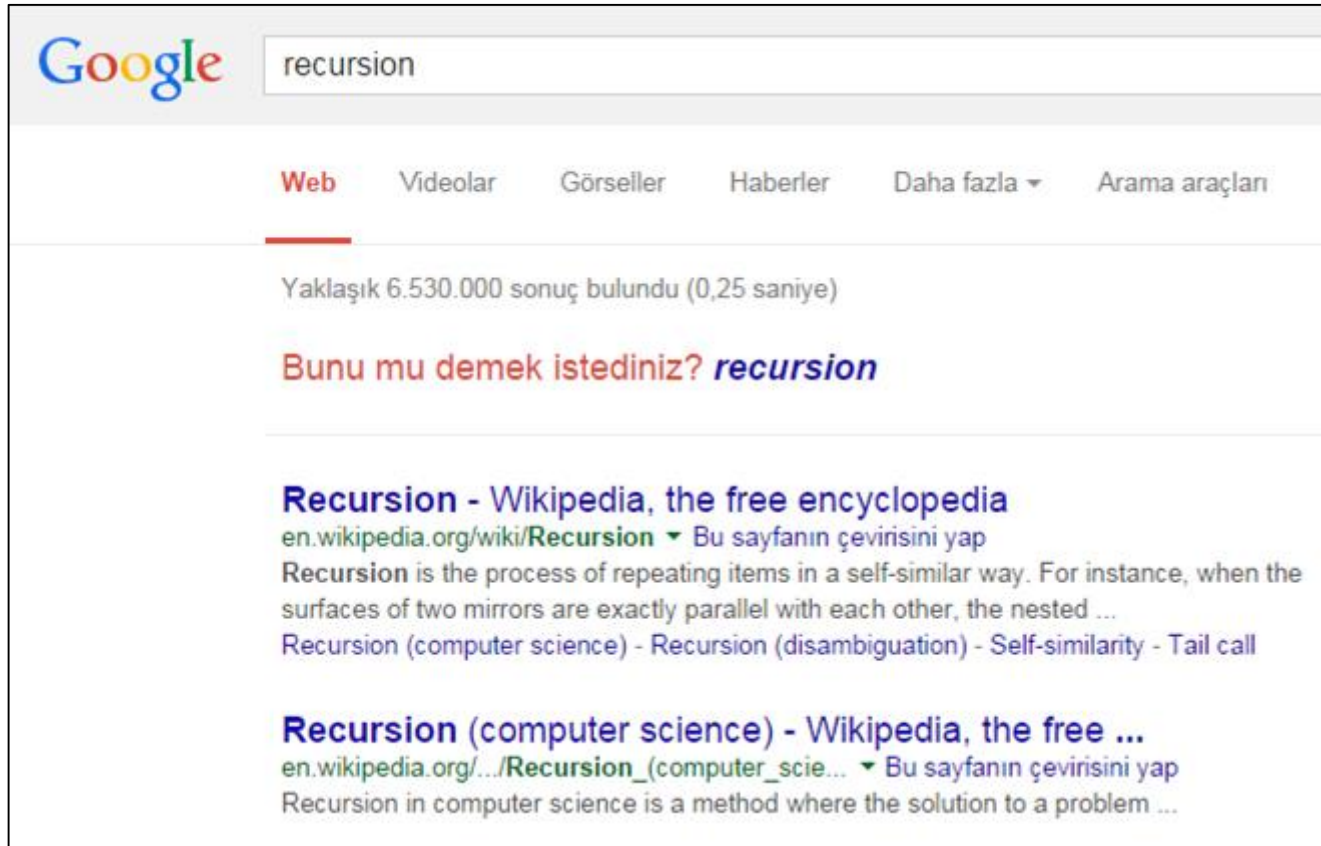
Executing: C:\Program Files (x86)\CodeBlocks/cb_console_runner.exe "C:\Users\Kaya Oğuz\Desktop\nonstatic.exe" (in C:\Users\Kaya Oğuz\Desktop)
Process terminated with status 0 (0 minute(s), 1 second(s))

static

- In the function `f()`, `int c` is declared each time the function is called.
 - Each time, a new variable `c` is defined (created).
 - If we make this variable static, it will only be created when the function `f()` is first called.
 - It will not be deleted and its value will be saved.
-



Recursion



Google recursion

Web Videolar Görseller Haberler Daha fazla ▾ Arama araçları

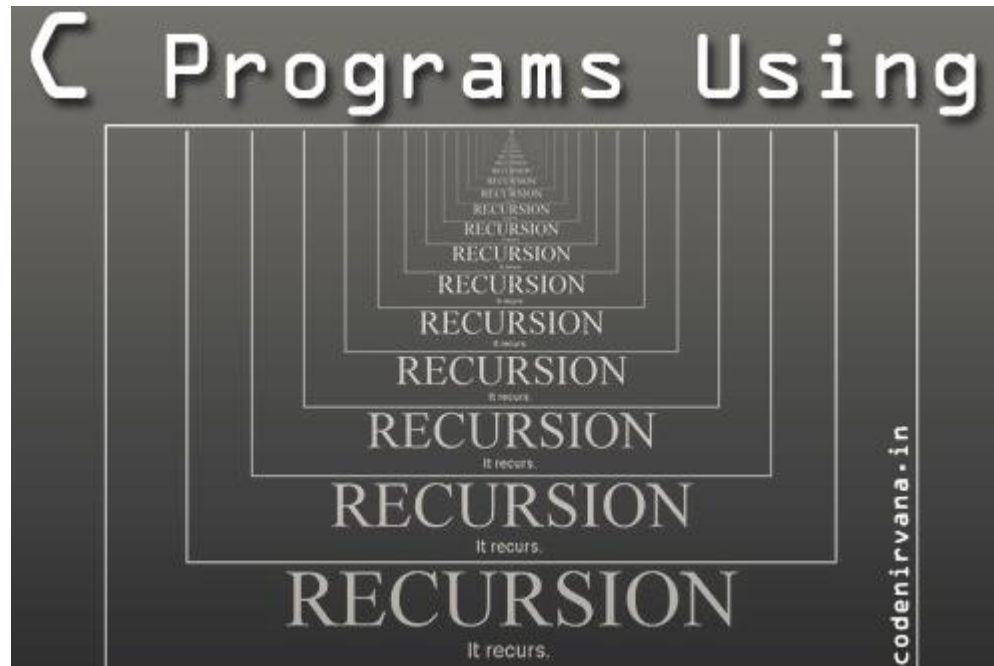
Yaklaşık 6.530.000 sonuç bulundu (0,25 saniye)

Bunu mu demek istediniz? *recursion*

Recursion - Wikipedia, the free encyclopedia
en.wikipedia.org/wiki/Recursion ▾ Bu sayfanın çevirisini yap
Recursion is the process of repeating items in a self-similar way. For instance, when the surfaces of two mirrors are exactly parallel with each other, the nested ...
[Recursion \(computer science\)](#) - [Recursion \(disambiguation\)](#) - [Self-similarity](#) - [Tail call](#)

Recursion (computer science) - Wikipedia, the free ...
[en.wikipedia.org/.../Recursion_\(computer_scie...](https://en.wikipedia.org/.../Recursion_(computer_scie...)) ▾ Bu sayfanın çevirisini yap
Recursion in computer science is a method where the solution to a problem ...

Recursion



Recursion

- If a function calls itself, it is recursive.

```
void recurse() {  
    printf("hello\n");  
    recurse();  
    return;  
}
```

- The function above calls itself on its second line.
 - The new instance of the function calls itself on its second line as well.
 - The new instance of the function calls itself on its second line as well.
 - ...
-

Was that an infinite loop?

- Since the function did not have a condition for NOT calling itself, it kept on calling itself, for infinity.
 - Let's rewrite it so that we can stop it.
 - We need a variable to get a hint of the number of recursions.
 - The variable can be passed as a parameter, or it can be global.
 - As a good programming practice, we stay away from globals as long as we can.
-



<global>

p1.c x file1.h x static.c x recursion01.c x

```
1  #include <stdio.h>
2
3  void recurse(int c) {
4      if(c != 5) {
5          printf("hello\n");
6          recurse(c + 1);
7      }
8      printf("The function with c = %d is complete.\n", c);
9      return;
10 }
11
12 int main() {
13     recurse(0);
14     return 0;
15 }
16
```

Logs & others

Code::Blocks x Search results x Cccc x Build log x Build messages x CppCheck x CppCheck messages

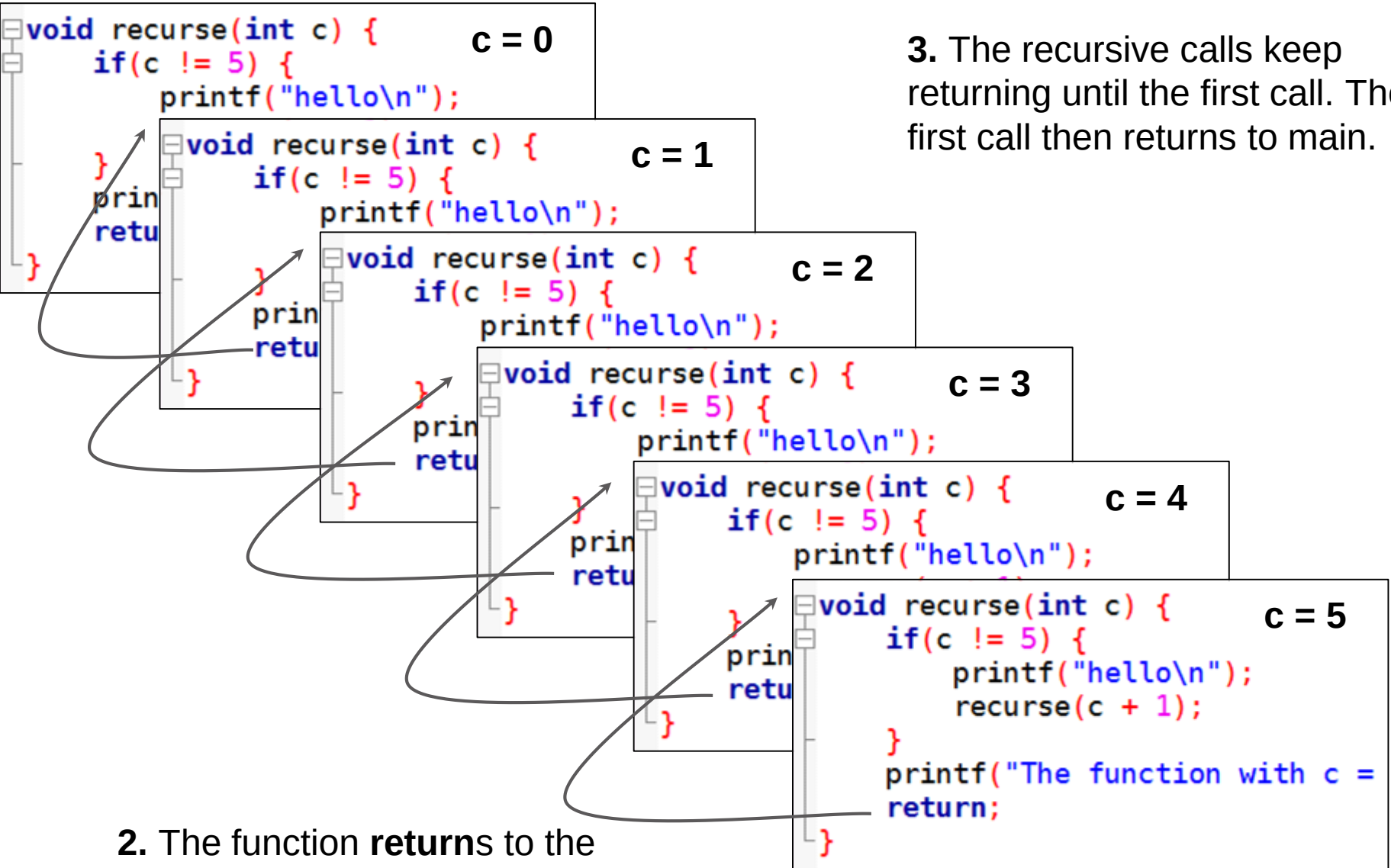
Executing: C:\Program Files (x86)\CodeBlocks\cb_console_runner.exe "C:\Users\Kaya Oğuz\Desktop\recursion01.exe" (in C:\Users\Kaya Oğuz\Desktop)

Process terminated with status 0 (0 minute(s), 13 second(s))

3. The recursive calls keep returning until the first call. The first call then returns to main.

2. The function **returns** to the point it was **called**. The **if** is complete and the **printf** is run. Then, this call also **returns**.

1. In this call, **if** returns false, and function **returns** to previous call.



Real-life recursion: Call waiting

- When you are talking on the phone to **A**, another call comes in. You hold the current caller and switch to the new incoming call: **B**.
 - Luckily, when you are talking to **B**, another call comes in, and you hold once again and switch to **C**.
 - Once you hang up with **C**, you return to your previous call, **B**.
 - Similarly, once you hang up with **B**, you return to **A**.
-

Real-life recursion: Call waiting

- In this example, what is the recursive function and its parameters?
 - It is very important you understand the way recursion works.
 - Similar to loops, you need a stopping condition.
 - In the code example we have also used starting conditions and the step.
 - **Homework:** How about using a static variable instead of passing the variable as a parameter?
-

Here is the output of the code

```
C:\Users\Kaya\Desktop\recursion01.exe
hello
hello
hello
hello
hello
The function with c = 5 is complete.
The function with c = 4 is complete.
The function with c = 3 is complete.
The function with c = 2 is complete.
The function with c = 1 is complete.
The function with c = 0 is complete.

Process returned 0 (0x0)    execution time : 0.009 s
Press any key to continue.
```

Notice the order of the output.

More Mathematics

- Let's write summation using iterative and recursive functions.

$$\sum_{i=1}^{i=N} i = 1 + 2 + 3 + \dots + (N - 1) + N$$

Reasoning

- The summation starts from 1 and continues until N.
 - Does it matter if it starts from 0?
 - The variable N can be a parameter to our function.
 - Since it is summation with integer values, the return type should be **int**.
 - We can write a loop (for/while) and sum at each step.
 - Finally, we can **return** the sum value.
-

Iterative summation

```
.c x  Untitled1.c x  sum-iterative.c x
1      #include <stdio.h>
2
3  int summation(int N) {
4      int i = 1;
5      int sum = 0;
6      for(i=1; i<=N; i++) { // notice the stopping condition
7          sum += i;
8      }
9      return sum;
10 }
11
12 int main() {
13     printf("sum(10) = %d\n", summation(10));
14     return 0;
15 }
16
```

Now, let's make it recursive

- To make the function recursive, we should be able to write the mathematical function in terms of itself as well.
- With summation, it is easy:

$$\sum_{i=1}^{i=N} = N + \sum_{i=1}^{i=N-1}$$

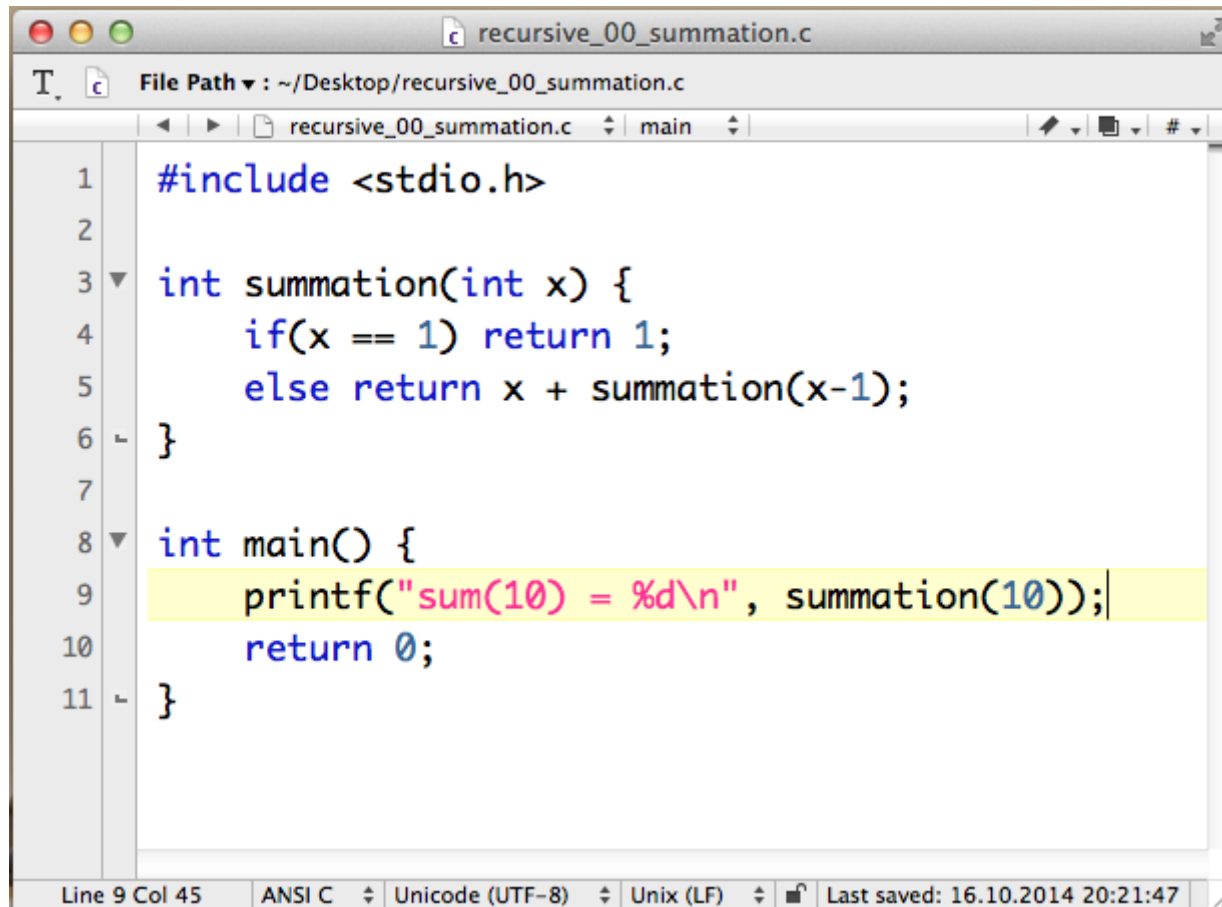
- To simplify:

$$f(x) = x + f(x-1)$$

A little more

- We did not write the stopping condition in the previous slide. But if $x == 1$, then it should not call itself, but return 1 instead.
 - So, the initial value is $x = N$, the step is $(x - 1)$ and the stopping condition is $(x == 1)$.
 - Let's go ahead and write it.
-

Recursive summation



The image shows a screenshot of a code editor window titled 'recursive_00_summation.c'. The editor displays the following C code:

```
1  #include <stdio.h>
2
3  int summation(int x) {
4      if(x == 1) return 1;
5      else return x + summation(x-1);
6  }
7
8  int main() {
9      printf("sum(10) = %d\n", summation(10));
10     return 0;
11 }
```

The code is written in ANSI C, using Unicode (UTF-8) encoding, and Unix (LF) line endings. The status bar at the bottom indicates the cursor is at Line 9, Column 45, and the file was last saved on 16.10.2014 at 20:21:47.

that looks elegant :)

```

int summation(int x) {
    if(x == 1) return 1;
    else return x + summation(x-1);
}

    int summation(int x) {
        if(x == 1) return 1;
        else return x + summation(x-1);
    }

        int summation(int x) {
            if(x == 1) return 1;
            else return x + summation(x-1);
        }

```

- The last call ($x = 1$) returns 1 to the call before it.
 - The call ($x = 2$) returns $x + 1$ (the 1 coming from the ($x = 1$) call).
 - The first call ($x = 3$) returns $x + 3$, which is the result, 6.
-

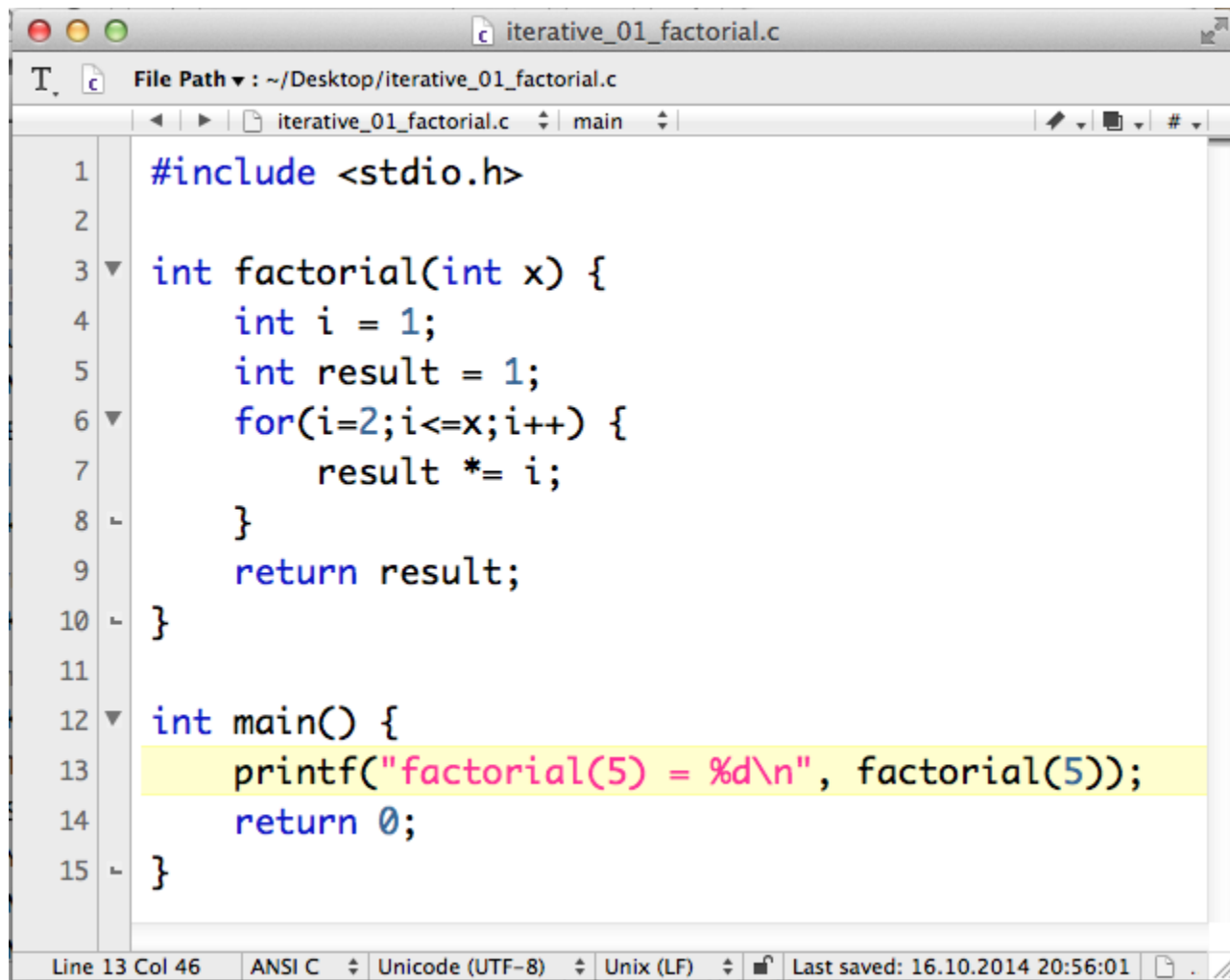
Factorial

- Another favorite function for recursion demonstration.
- Here is a recap.

$$n! = n \times (n - 1) \times \cdots \times 1$$

- $0! = 1$
 - $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
 - Let's go with iterative first.
-

Iterative Factorial



The image shows a screenshot of a code editor window titled "iterative_01_factorial.c". The editor displays the following C code:

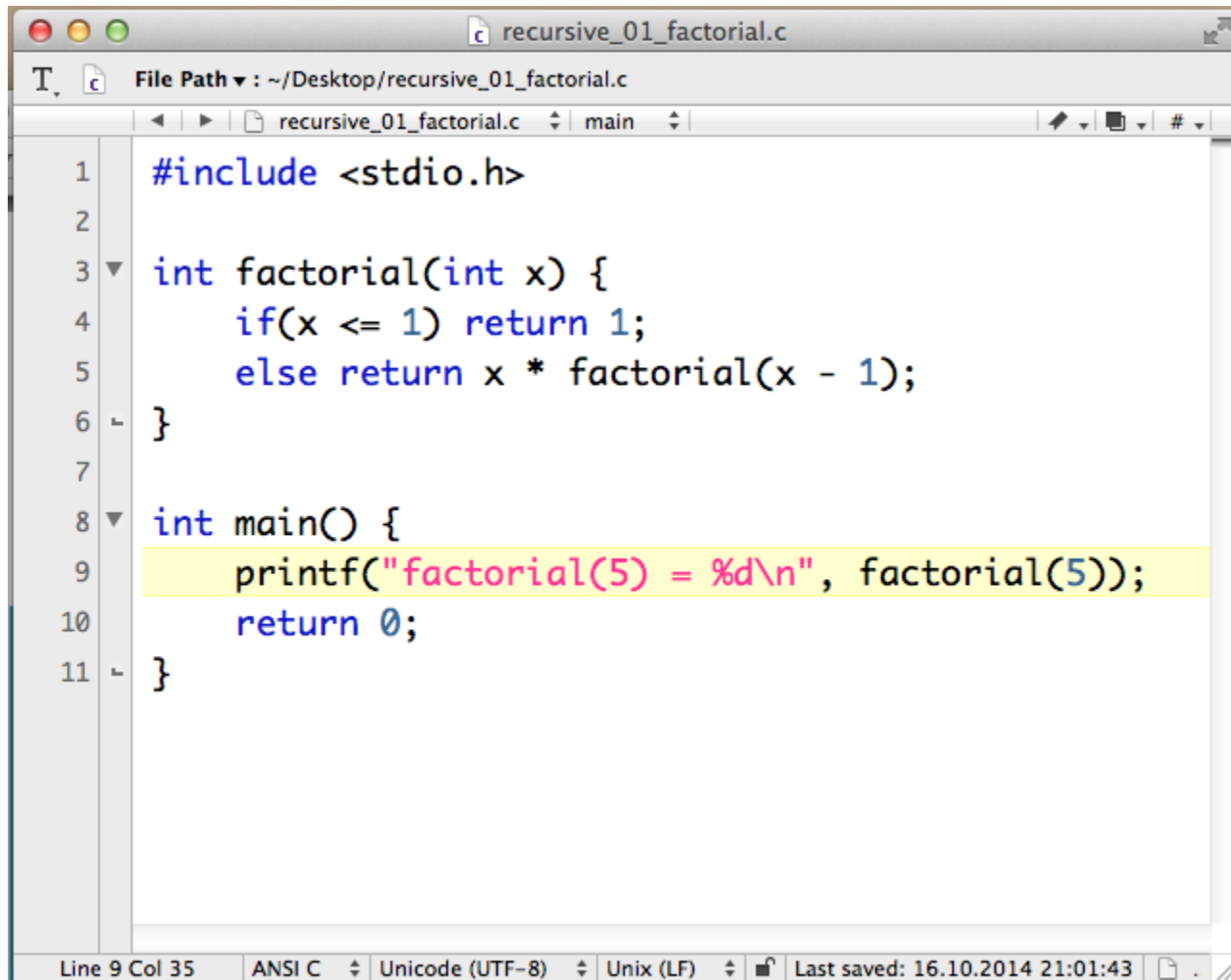
```
1  #include <stdio.h>
2
3  int factorial(int x) {
4      int i = 1;
5      int result = 1;
6      for(i=2; i<=x; i++) {
7          result *= i;
8      }
9      return result;
10 }
11
12 int main() {
13     printf("factorial(5) = %d\n", factorial(5));
14     return 0;
15 }
```

The code is written in C and calculates the factorial of 5 using an iterative approach. The `factorial` function takes an integer `x` and returns its factorial. The `main` function calls `factorial(5)` and prints the result. The line `printf("factorial(5) = %d\n", factorial(5));` is highlighted in yellow. The editor's status bar at the bottom indicates "Line 13 Col 46", "ANSI C", "Unicode (UTF-8)", "Unix (LF)", and "Last saved: 16.10.2014 20:56:01".

Recursive Factorial

- Can you define factorial function recursively?
 - Well, it is very similar to summation.
 - $f(x) = x * f(x-1)$
 - Of course, we need a stopping condition.
 - Zero (0) is dangerous. We should stop at 1.
 - Here is your recursive function.
-

Recursive Factorial



```
recursive_01_factorial.c
File Path: ~/Desktop/recursive_01_factorial.c
recursive_01_factorial.c | main |
1  #include <stdio.h>
2
3  int factorial(int x) {
4      if(x <= 1) return 1;
5      else return x * factorial(x - 1);
6  }
7
8  int main() {
9      printf("factorial(5) = %d\n", factorial(5));
10     return 0;
11 }
```

Line 9 Col 35 ANSI C Unicode (UTF-8) Unix (LF) Last saved: 16.10.2014 21:01:43

Power function

- Let's directly think about a recursive power function. (Iterative version is your homework.)
- The power function has a base and a power.
- We multiply the base as many times as the power.

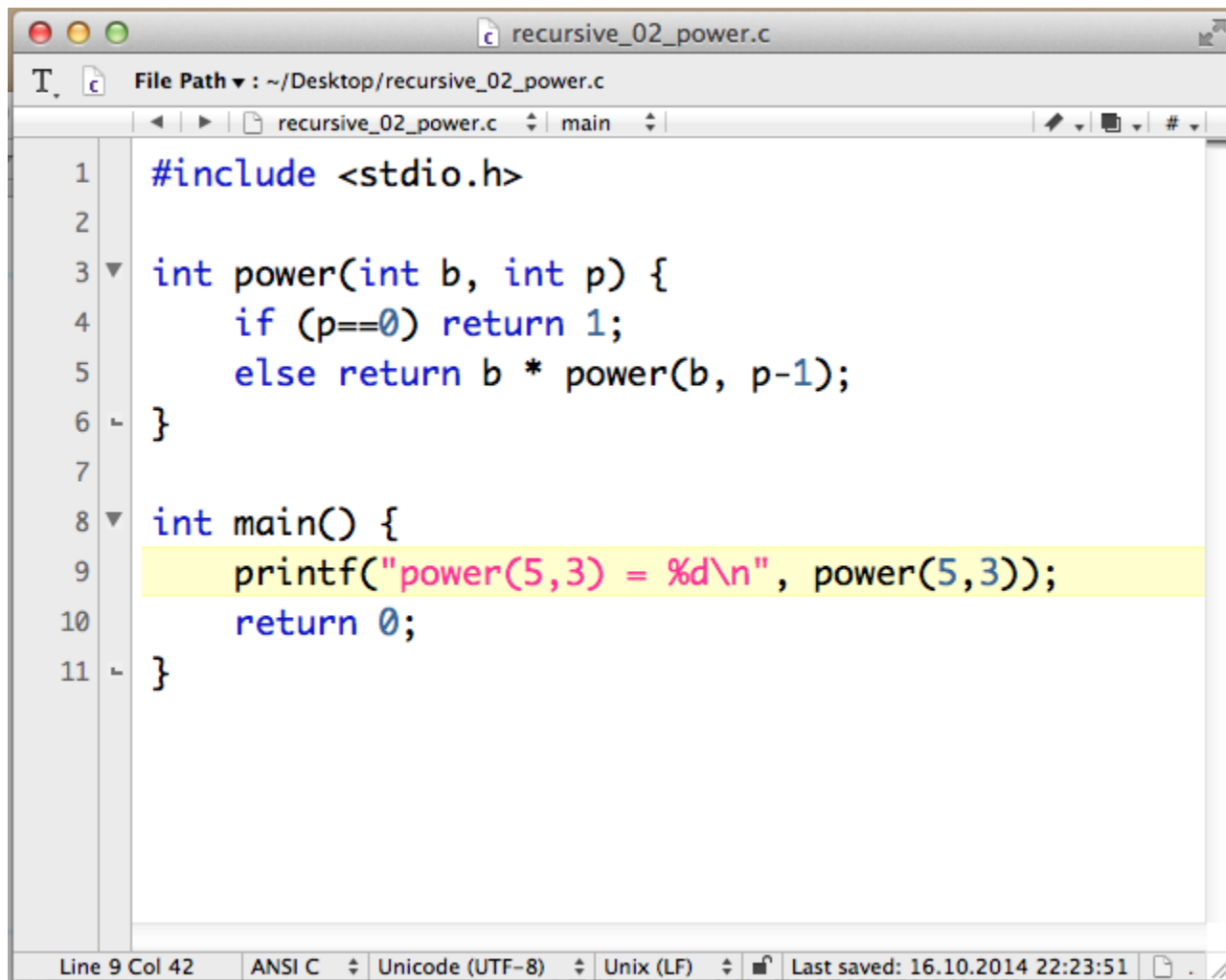
$$m^n = m \times m \times \cdots \times m$$

$$5^3 = 5 \times 5 \times 5$$

Identify the repetition

- The base is repeated.
 - The repetition is done power times: this is our stopping condition.
 - We need two parameters. One for the base, one for keeping track of power.
 - $\text{power}(b, p) = b * \text{power}(b, p-1)$
 - The recursion is now clear. We need to check for the value of p (power) in the function to stop.
-

Recursive Power



The image shows a screenshot of a code editor window titled "recursive_02_power.c". The editor displays the following C code:

```
1  #include <stdio.h>
2
3  int power(int b, int p) {
4      if (p==0) return 1;
5      else return b * power(b, p-1);
6  }
7
8  int main() {
9      printf("power(5,3) = %d\n", power(5,3));
10     return 0;
11 }
```

The code implements a recursive function to calculate the power of a base b raised to an exponent p . The base case is when $p=0$, returning 1. The recursive case returns b multiplied by the result of $\text{power}(b, p-1)$. The `main` function calls `power(5,3)` and prints the result.

The status bar at the bottom indicates: Line 9 Col 42, ANSI C, Unicode (UTF-8), Unix (LF), and Last saved: 16.10.2014 22:23:51.

Fibonacci Numbers

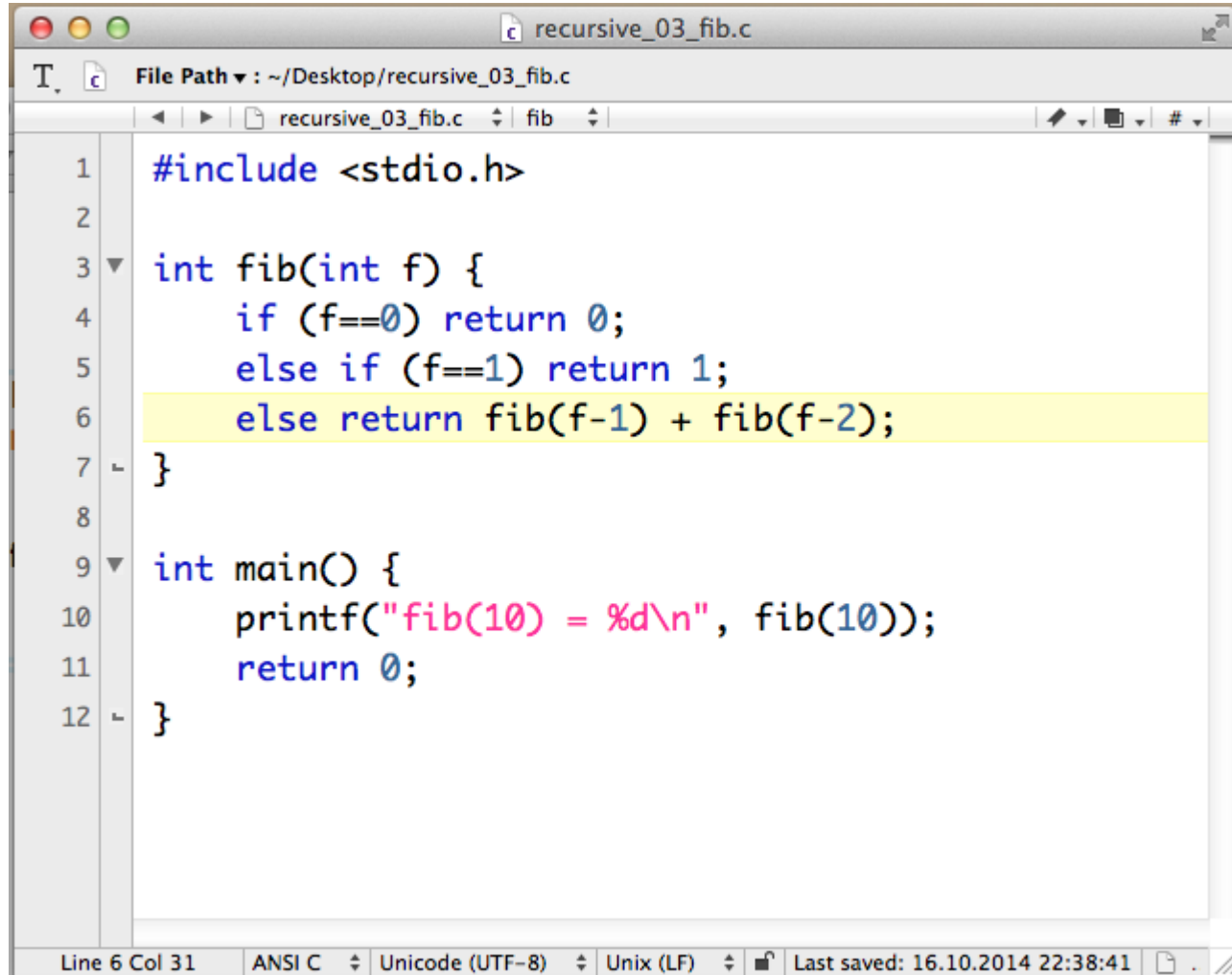


- The famous Fibonacci numbers...
- Each Fibonacci number is the sum of the previous two Fibonacci numbers.
- So, $F(3) = F(2) + F(1)$
- Formally:
$$F_n = F_{n-1} + F_{n-2}$$
- Of course, $F(0) = 0$ and $F(1) = 1$.
- Read more about it on Wikipedia
 - http://en.wikipedia.org/wiki/Fibonacci_number

Recursive Fibonacci

- I already gave you the recursive function definition.
 - The stopping condition for $F(x)$ is that $x == 0$ or $x == 1$. Then we return 0 or 1 respectively.
 - Otherwise, we call the function again using $F(x-1)$ and $F(x-2)$.
 - Alright, I'll write it. But the iterative version is your homework.
-

Recursive Fibonacci



The image shows a screenshot of a code editor window titled "recursive_03_fib.c". The editor displays the following C code:

```
1  #include <stdio.h>
2
3  int fib(int f) {
4      if (f==0) return 0;
5      else if (f==1) return 1;
6      else return fib(f-1) + fib(f-2);
7  }
8
9  int main() {
10     printf("fib(10) = %d\n", fib(10));
11     return 0;
12 }
```

The code is written in ANSI C, using Unicode (UTF-8) encoding, and Unix (LF) line endings. The status bar at the bottom indicates the current position is Line 6 Col 31, and the file was last saved on 16.10.2014 at 22:38:41.

Greatest Common Divisor

- Given two numbers, say, 111 and 259, what is the greatest integer that divides them both.
 - Do you remember the old school method?
 - Here is a more practical way.
-

Greatest Common Divisor

$$\text{gcd}(x, y) = \begin{cases} x & \text{if } y = 0 \\ \text{gcd}(y, \text{remainder}(x, y)) & \text{if } y > 0 \end{cases}$$

- The algorithm is easy. If y (second parameter) is not 0, then call gcd again, this time the second parameter becomes the first parameter and the second parameter is the remainder of x / y .
 - That is $x \% y$
 - This is your homework.
-

More recursive

- Such as in loops, you need a stopping condition in recursive function calls. Otherwise you get an infinite loop. We don't want infinite loops.
-

Discussion: iteration vs. recursion

- Which one to promote?
 - Consider the memory (overhead: function call stack)
 - Consider the CPU (overhead: function calls)
 - Consider the problem's nature and the programmer's solution approach
-

C preprocessor

- The C preprocessor is the macro preprocessor for C (and C++ as well).
 - It is a separate program invoked by the C compiler.
 - Its directives come after the # sign.
 - Commonly used directives are:
 - include and define
 - https://en.wikipedia.org/wiki/C_preprocessor
-

Having fun?

just checking *



A few more homework assignments

- Homework: Try entering invalid data, like floating point numbers or a character and check the output.
 - Another homework: Write a program that will use %c, %f, %lf
 - One more: Research formatting using printf. For example, can I print a variable $x = 2$ as 02?
-

Extra homework assignments

1. Write a function that halves an integer. Since the integer can be odd, be careful with the return type.
 2. Write a function that takes power using two parameters, one for the base, one for the power. For example `power(2, 5)` should return 32 ($=2^5$).
 3. The program we worked on is not complete. Re-write the main that will call the appropriate function until the user decides to exit. You can ask for input in the main.
-

Credits

- C How To Program, 7/e
Deitel & Deitel, Prentice Hall, ISBN10:
013299044X, ISBN13: 978-0132990448
 - The C Programming Language, 2nd edition,
Brian W. Kernighan, Dennis M. Ritchie,
Prentice Hall, ISBN13:007-6092003106
 - Lecture slides by Kaya Oğuz
 - Lecture slides by Senem Kumova Metin
-